

Principal Components Analysis

A self-study companion to the lecture

Everything the slides say, in the order we said it in class — with the pictures, the full derivation, a worked example done to the last decimal, and seven practice questions with solutions.

What is inside

§1–§4	why we chase variance, and the full constrained-optimisation proof
§5–§9	eigenvectors, the SVD link, low rank, and what to do when D is huge
§10–§12	choosing components, the four-step recipe, a worked example
§13–§15	traps, a one-page card, and seven solved practice questions

Prepared as a companion to the Lecture 13 slide deck. The slide deck stays the authority on what is examinable; this document exists to make it easy to follow.

How to read this

This document follows the Lecture 13 slides, but it takes the route we actually took in class: start from *why* we care about variance at all, build the constrained-optimisation view, and only then turn to the linear-algebra machinery the slides spend most of their time on.

- **Read the one-line summary under each section heading first.** If you read only the section titles and those one-liners, you already have the shape of the whole lecture.
- **The coloured boxes each mean one thing.** ■ navy is the idea to remember; ■ amber is an everyday picture; ■ slate is something we said in the class; ■ teal is a fully worked example; ■ purple is a point that surprises most people; ■ rust is a trap to avoid.
- **Nothing is skipped.** Where the slides write “it can be shown that”, this document shows it.
- **Section 15 is the practice set.** Seven questions, each of a different type, each with a full solution and a note on how to attack that type of question.

Symbols used throughout

Symbol	Meaning	Symbol	Meaning
N	number of data points (rows)	λ_d	the d th eigenvalue of C
D	number of original features (columns)	w_d	the d th eigenvector, a unit vector
M	number of components we keep, $M < D$	W	$D \times M$ matrix of kept eigenvectors
X	the centred data matrix, $N \times D$	Z	the scores, $Z = XW$, size $N \times M$
x_i	one data point, a row of X	σ_d	singular value, or std. dev. (see below)
C	covariance matrix, $\frac{1}{N} X^\top X$	$\tilde{x}$	a point rebuilt from its scores

A note on notation. Different sources write the same objects differently, and this trips people up more than the mathematics does. The slide deck writes S for the covariance matrix and puts data points in the *columns* of X . In class we wrote C and put data points in the *rows*, like a spreadsheet. Exam papers often follow the statistics convention and write Σ for the covariance of a random vector X , with α for the direction. **All three are the same object.** This document uses the spreadsheet convention (C , rows) because it is the one we derived everything in; §3 gives the exact dictionary between them, and §15 switches to Σ and α so that the practice questions look like the real thing.

Contents

1	Start here: pick the better line	4
2	Why variance is the thing we chase	4
3	The objects PCA works with — and the three notations	6
4	PCA is a constrained optimisation problem	8
5	From the covariance matrix to the directions	11
6	The same picture through the SVD	13
7	Low-rank approximation: the Eckart–Young theorem	14
8	Getting the eigenvectors in practice	16
9	When D is huge: wide data and the Gram trick	17
10	How many components to keep, and how to read them	20
11	The recipe: four steps, end to end	23
12	A worked example, start to finish	26
13	Traps, limits, and honest caveats	29
14	The one-page card	32
15	Practice set: seven questions, fully solved	33

1 Start here: pick the better line

Given a cloud of points and two candidate lines, the whole of PCA is choosing the line the points spread out on the most.

Before any definitions, do the thing itself.

Figure 1 shows the same ninety data points twice. Each panel picks one line through the middle of the cloud and slides every point onto it along the shortest path. That sliding is called **projection**, and the coloured dots on the line are the projected points.

Look only at the coloured dots. On the left they are spread out along a long stretch of the line. On the right they are bunched into a short segment.

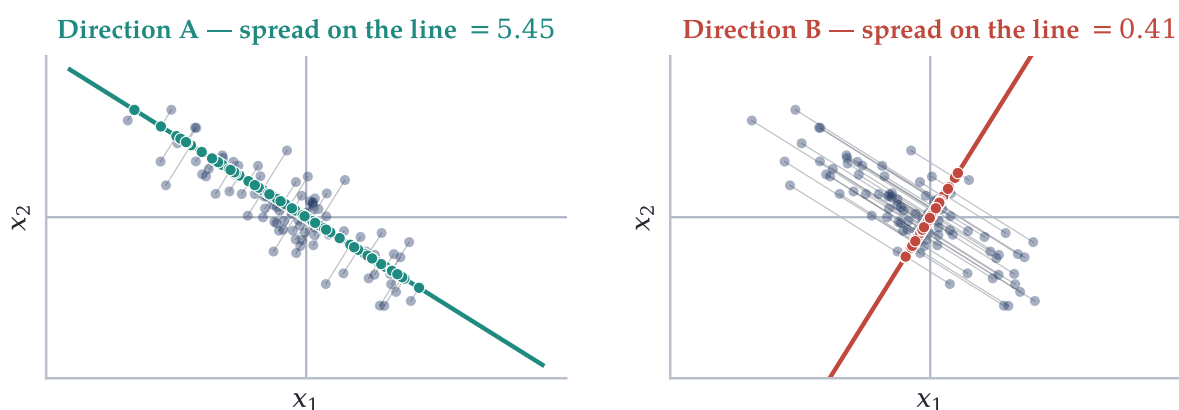


Figure 1: Two candidate lines, same data. Grey threads show each point dropping onto the line. *How to read it:* compare how far apart the coloured dots end up. Direction A keeps the points distinct; direction B squashes different points on top of each other.

If you had to describe each point by a *single* number — its position along the line — which line would you choose?

Direction A, obviously. On direction B, points that were far apart in the original cloud land almost on top of each other, so that single number cannot tell them apart any more. On direction A the points stay distinguishable. The number that measures “how far apart the dots ended up” is the **variance**, printed in each panel’s title.

You have just done PCA by eye.

The whole method in one sentence

PCA searches over every possible direction, computes the variance of the data after projecting onto it, and keeps the directions with the highest variance. Everything else in this document is the machinery that does that search efficiently, plus the reason it works.

We went from two dimensions to one here. In real work you go from a hundred columns to five, or from twenty thousand to fifty. The principle does not change; only the bookkeeping gets bigger.

2 Why variance is the thing we chase

Every model says $y = f(x) + \varepsilon$; a column of x that never changes can never explain a change in y ; so “how much does this column change” is the right question, and variance is its answer.

The first section showed *what* PCA does. This section says *why* variance — and not some other quantity — is the thing worth maximising. It is worth a few minutes, because once you see it, everything from the covariance matrix to the eigenvalue ordering feels inevitable.

2.1 Every model you have ever built has the same skeleton

Whatever the model — linear regression, a decision tree, a neural network — you are writing the same statement:

$$y = f(x) + \varepsilon. \quad (1)$$

The target y is driven by the features x through some relationship f , plus an error ε for everything you did not, or could not, measure. A linear model makes f a straight line. A tree makes f a set of splits. A network makes f a stack of transformations. The skeleton is identical.

Now read the equation as a statement about *change* rather than about values:

a change in y comes either from a change in x , or from a change in the error.

2.2 The statistic that measures change is variance

Ask what tool measures “how much does this thing change”. Not the mean — the mean says where things sit, not how much they move. The answer is the variance:

$$\text{Var}(x) = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2, \quad \mu = \frac{1}{N} \sum_i x_i. \quad (2)$$

This is not a new idea dressed up. You have already used exactly this split before, under different names.

You have met this split before

In the analysis of variance you write $\text{TSS} = \text{SST} + \text{SSE}$: the total variation in y equals the part your factors explain plus the part they do not. The F test is the ratio of the two, “known variance over unknown variance”. And $R^2 = 1 - \text{SSE}/\text{TSS} = \text{SST}/\text{TSS}$ is simply *the share of the variance in y that your features account for*.

All three are the same accounting exercise: split the movement in y into an explained part and an unexplained part. Variance is the currency in every one of them.

2.3 The one surprising move: PCA never looks at y

Here is where PCA departs from everything above. It throws y away.

Suppose you are predicting salary from four columns: age, experience, number of certifications, number of trainings. PCA is handed *only* the four columns. It never sees a single salary.

Why is that reasonable? Because of the assumption we just wrote down: if x does not change, y cannot change because of x . So a column with almost no variance carries almost no ability to explain anything — whatever the target happens to be. You can rank the usefulness of directions in x before you ever look at y .

In Figure 2 the last two columns are near-constant — almost every employee has zero certifications and zero trainings. Their variance is essentially zero, so they are dead weight. Dropping them costs nothing.

That leaves age and experience. These two are a different problem: they are highly correlated. Older employees generally have more experience, so the two columns largely repeat each other. A standard feature-selection method would delete one of them. PCA does something better



Figure 2: The four columns of the salary table. *How to read it:* certifications and trainings are almost always zero, so their bars are invisible next to the others. Nothing that never moves can explain something that does.

— it builds a single new direction that carries what both columns were saying, and a second direction, at right angles to the first, that carries whatever little is left.

What we said in class

“PCA doesn’t get bothered by what you have in y . It only takes your x input features. We are only caring about how x changes, without even bothering about what your y is.” That is the defining feature of PCA, and it is also its main weakness — see §13, where we look at a case where the direction PCA discards is exactly the one that predicts y .

2.4 Three names for the same operation

Because PCA replaces D old columns with M new ones, people describe it in three different vocabularies. All three are correct.

Called	Because	Compare with the usual approach
dimension reduction	D columns go in, $M < D$ columns come out	the honest description of what happens
feature selection	you are choosing which directions survive	usual selection needs a model first, then feature importance, then a cut. PCA selects <i>before</i> any model is built
feature engineering	each new column is a fresh linear mix $z_1 = a_1x_1 + \dots + a_Dx_D$ of the old ones	same spirit as building loan/income by hand, except PCA finds the weights for you

And it throws in a fourth property for free: the new columns are mutually uncorrelated. We prove that in §10; Figure 9 shows it happening.

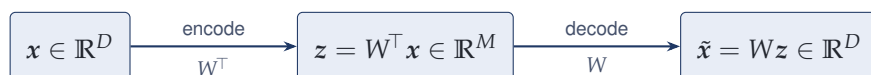
Section recap. Models explain change in y using change in x . Variance measures change. A feature that does not vary cannot explain anything, so we can rank directions in x without ever looking at y . PCA does exactly that, and the new directions it produces are uncorrelated as a bonus.

3 The objects PCA works with — and the three notations

Four objects: the data X , a direction w , the score z , and the rebuilt point $\tilde{x}$. The only real difficulty is that three sources write them three different ways.

3.1 The four objects

- $x \in \mathbb{R}^D$ — one data point, with D original features. All N points stacked as rows give the data matrix X , size $N \times D$. We always assume X has been **centred**: the mean of every column has been subtracted, so each column of X averages to zero.
- W — a $D \times M$ matrix whose columns $w_1, \dots, w_M$ are the directions we keep. Each is a *unit* vector, $w_m^\top w_m = 1$, and they are mutually perpendicular.
- $z \in \mathbb{R}^M$ — the **scores**, or **code**, of a point: $z = W^\top x$. Component z_m is how far along direction w_m the point sits. For the whole dataset at once, $Z = XW$.
- $\tilde{x} \in \mathbb{R}^D$ — the point **rebuilt** from its scores: $\tilde{x} = Wz = WW^\top x$. This lives back in the original D -dimensional space, but only on the M -dimensional slice that W spans.



The covariance matrix ties them together:

$$C = \frac{1}{N} X^\top X \in \mathbb{R}^{D \times D}, \quad C_{jk} = \frac{1}{N} \sum_{i=1}^N x_{ij} x_{ik}. \quad (3)$$

Entry C_{jj} is the variance of column j . Entry C_{jk} is the covariance between columns j and k — how much they move together. C is symmetric, and it is the only thing about the data that PCA ever uses.

Divide by N , and centre first

Two details that cost marks.

One. Equation (3) divides by N , not by $N - 1$. The $N - 1$ version is the unbiased estimator for a *sample*, and it is what numpy and R use by default. The slide deck — and therefore this course — uses N . Use N .

Two. The formula $\frac{1}{N} X^\top X$ is only the covariance if X is already centred. If you skip the centring you get the matrix of raw second moments, which is a different matrix with different eigenvectors. Centre first, every time.

3.2 The dictionary between the three notations

This is the single most common source of confusion when you move between the slides, the class notes and a question paper. They describe identical objects.

	This document & the class	The slide deck	A typical exam paper
data laid out as	rows = points	columns = points	a random vector X
size of X	$N \times D$	$D \times N$	—
covariance written	$C = \frac{1}{N} X^\top X$	$S = \frac{1}{N} X X^\top$	$\Sigma = \text{Cov}(X)$
a direction written	w	b	α
the eigen-equation	$Cw = \lambda w$	$Sb = \lambda b$	$\Sigma \alpha = \lambda \alpha$
the projection	$Z = XW$	$z = B^\top x$	$Y_1 = \alpha^\top X$
the rebuilt point	$\tilde{X} = XWW^\top$	$\tilde{x} = Bz$	—

Transposing X swaps $X^\top X$ and $XX^\top$; that is the whole difference between column one and column two. Column three drops the data matrix entirely and speaks about the population covariance of a random vector, which is the cleanest way to state the theory. Every result in this document holds in all three.

Σ means two different things

The slide deck uses the letter Σ for the diagonal matrix of *singular values* in the SVD, $X = U\Sigma V^\top$. Exam papers use the same letter for the *covariance matrix*. These are unrelated objects that happen to share a symbol.

In this document: C is always the covariance matrix, and Σ is always the singular-value matrix — except in §15, where we switch to the exam’s convention and say so explicitly at the top.

The lower-case σ is overloaded too, and unavoidably so. Wherever the SVD is in play — §5’s comparison table, §6, §7, the card and Question 7 — σ_d is the d th *singular value*. Everywhere standardising is discussed, including Question 3, σ_d is the *standard deviation* of column d . Both appear in §15, so read the surrounding sentence rather than the symbol alone.

Section recap. Centre the data. Build $C = \frac{1}{N}X^\top X$ (divide by N). Encode with $z = W^\top x$, decode with $\tilde{x} = Wz$. When a source looks unfamiliar, check whether its data points are rows or columns — that explains almost every apparent disagreement.

4 PCA is a constrained optimisation problem

Maximise $w^\top Cw$ subject to $w^\top w = 1$; the Lagrangian turns that into $Cw = \lambda w$, and the maximum value is λ itself.

This is the centre of the lecture. Everything before it is motivation and everything after it is machinery. Six lines of algebra take us from “find the direction of most spread” to “find the top eigenvector of the covariance matrix”, and they explain *why* eigenvectors appear in a problem that never mentioned them.

4.1 Setting up the problem

We want a direction $w \in \mathbb{R}^D$. Project the centred data onto it:

$$z = Xw, \quad z_i = x_i^\top w. \quad (4)$$

Each z_i is one number: where point i lands on the line through w .

Why the direction must have length one. Without a constraint the problem is broken. Double the length of w and every z_i doubles, so the variance quadruples — you could make it as large as you like by stretching w , without pointing it anywhere better. That is cheating: we care about the *direction*, not the length. So we demand

$$w^\top w = 1. \quad (5)$$

Recall from the vectors lecture that the length of a vector is the square root of its dot product with itself, $\|w\| = \sqrt{w^\top w}$. Squaring both sides of $\|w\| = 1$ gives $w^\top w = 1$, which is the form we want because it is polynomial and easy to differentiate.

4.2 The objective, in three lines

The projected values already have mean zero, because X was centred and projection is linear. So their variance is just the average of the squares:

$$\text{Var}(z) = \frac{1}{N} \sum_{i=1}^N z_i^2 = \frac{1}{N} z^\top z \quad \text{a sum of squares is a dot product} \quad (6)$$

$$= \frac{1}{N} (Xw)^\top (Xw) \quad \text{substitute } z = Xw \quad (7)$$

$$= w^\top \left(\frac{X^\top X}{N} \right) w = w^\top C w. \quad \text{regroup; the bracket is } C \quad (8)$$

Equation (8) is worth pausing on. The left side is a statement about the projected data. The right side involves only C and w — the data has vanished into the covariance matrix. Every direction w now gets a single score, $w^\top C w$, computed without touching the raw data again.

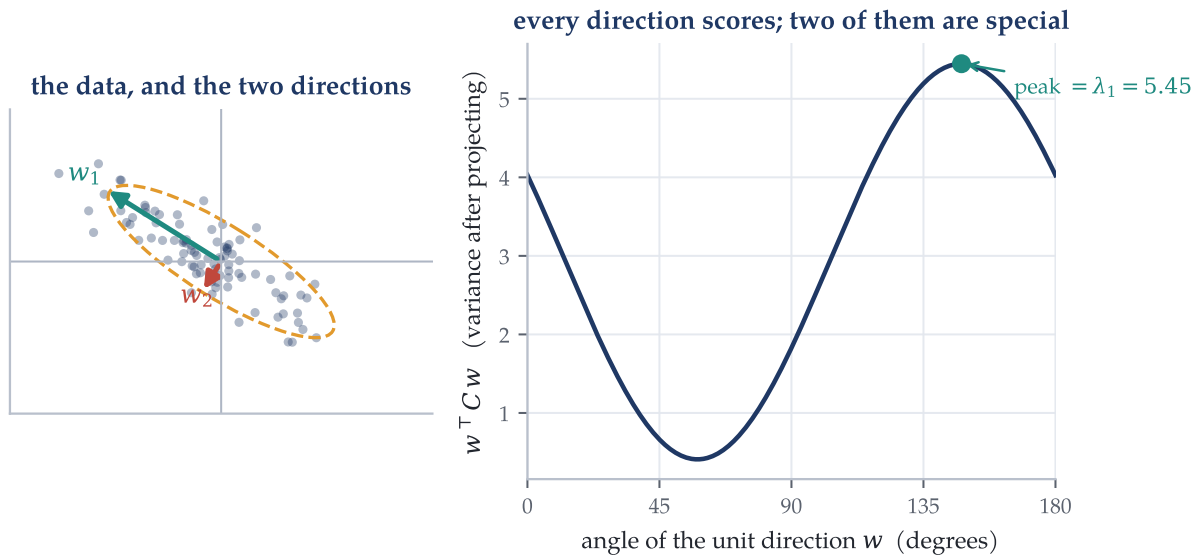


Figure 3: The score $w^\top C w$ for every unit direction in the plane. *How to read it:* sweep the angle from 0° to 180° and the score traces one smooth hump. Its peak is λ_1 and sits exactly along w_1 ; its dip is λ_2 , exactly 90° away.

So the problem is

$$\boxed{\max_w w^\top C w \quad \text{subject to} \quad w^\top w = 1.} \quad (9)$$

Maximise something, subject to an equality constraint. That is a constrained optimisation problem, and we already know the tool for it.

4.3 The Lagrangian, and where the eigenvectors come from

Write the constraint in the standard form “expression = 0”: $w^\top w - 1 = 0$. Then

$$\mathcal{L}(w, \lambda) = w^\top C w - \lambda(w^\top w - 1). \quad (10)$$

Differentiate with respect to w . Both terms are quadratic forms, and for a symmetric matrix A the gradient of $w^\top A w$ is $2Aw$. Here C is symmetric and the identity is symmetric, so

$$\frac{\partial \mathcal{L}}{\partial w} = 2Cw - 2\lambda w = \mathbf{0}. \quad (11)$$

Divide by two and move one term across:

$$Cw = \lambda w \quad (12)$$

Look at what happened. We asked a question about spread and got back the definition of an eigenvector. Any direction that could possibly maximise the projected variance must be an eigenvector of the covariance matrix. Nothing else is even a candidate.

4.4 The payoff: the maximum variance is the eigenvalue

We know the answer is one of the eigenvectors — but which one? Substitute (12) back into the objective (8):

$$\text{Var}(z) = w^T C w = w^T (\lambda w) = \lambda w^T w = \lambda \cdot 1 = \lambda. \quad (13)$$

The result to remember

The variance you get by projecting onto an eigenvector *equals its eigenvalue*. So to maximise the variance, pick the eigenvector with the largest eigenvalue. To get the best M directions, take the top M eigenvalues.

$$\text{maximum variance} = \lambda_{\max}, \quad \text{best direction} = w_{\max}.$$

This also explains the ordering convention $\lambda_1 \geq \lambda_2 \geq \dots$: it is a ranking of the directions by how much of the data's spread each one captures. And it is why a scree plot works at all.

That single line, (13), answers a whole family of questions at once. “What is the maximum variance of the first principal component?” λ_1 . “What is the variance along the second?” λ_2 . No computation required.

The Lagrange multiplier *is* the variance

A question that came up in class: the multiplier λ in (10) is a penalty term — what does it have to do with an eigenvalue?

It turns out they are the same number. In (12) the multiplier appears as the eigenvalue, and in (13) the same number turns out to be the maximum variance. So all three readings are one quantity:

Read as	It is	Where you see it
a Lagrange multiplier	the price of the constraint $\ w\ = 1$	(10)
an eigenvalue	the stretch factor of C along w	(12)
a variance	the spread of the data along w	(13)

Keep the roles straight, though: the *principal component* is the eigenvector w — a direction. The *eigenvalue* λ is the variance along it — a number.

4.5 The second component, and all the rest

The same argument gives every later component. For the second, add one more constraint — it must be perpendicular to the first:

$$\max_{w_2} w_2^T C w_2 \quad \text{subject to} \quad w_2^T w_2 = 1, \quad w_2^T w_1 = 0. \quad (14)$$

Now the Lagrangian carries two multipliers: $\mathcal{L} = \mathbf{w}_2^\top C \mathbf{w}_2 - \lambda(\mathbf{w}_2^\top \mathbf{w}_2 - 1) - \mu \mathbf{w}_2^\top \mathbf{w}_1$. Setting the gradient to zero gives $2C\mathbf{w}_2 - 2\lambda\mathbf{w}_2 - \mu\mathbf{w}_1 = \mathbf{0}$. Multiply on the left by $\mathbf{w}_1^\top$ and use $\mathbf{w}_1^\top \mathbf{w}_2 = 0$, $\mathbf{w}_1^\top \mathbf{w}_1 = 1$ and $\mathbf{w}_1^\top C = \lambda_1 \mathbf{w}_1^\top$:

$$2\lambda_1 \underbrace{\mathbf{w}_1^\top \mathbf{w}_2}_{=0} - 2\lambda \underbrace{\mathbf{w}_1^\top \mathbf{w}_2}_{=0} - \mu \underbrace{\mathbf{w}_1^\top \mathbf{w}_1}_{=1} = 0 \quad \implies \quad \mu = 0. \quad (15)$$

With $\mu = 0$ the condition collapses back to $C\mathbf{w}_2 = \lambda\mathbf{w}_2$: an eigenvector again, and the best available one is the eigenvector of the second largest eigenvalue. Repeat for $\mathbf{w}_3$, and so on.

We never had to *impose* that the components be perpendicular, incidentally. C is symmetric, and the spectral theorem guarantees a symmetric matrix has a full orthonormal set of eigenvectors. Perpendicularity comes free.

This derivation is the most reusable thing in the lecture. It may be asked for in general form (“show that α satisfies $\Sigma\alpha = \lambda\alpha$ ”), or with a specific matrix, or for the second component — but it is the same six lines every time. Get it to the point where you can write it from memory and also spot a fault in someone else’s version; Question 2 in §15 trains the second of those.

Section recap. PCA is $\max \mathbf{w}^\top C \mathbf{w}$ s.t. $\mathbf{w}^\top \mathbf{w} = 1$. The Lagrangian gives $C\mathbf{w} = \lambda\mathbf{w}$, so the candidates are exactly the eigenvectors of C . Substituting back gives $\text{Var} = \lambda$, so the best direction is the top eigenvector and the maximum variance is the top eigenvalue.

5 From the covariance matrix to the directions

C is symmetric, so it always has a full set of perpendicular eigenvectors with real eigenvalues; their sum is the total variance in the data.

Section 4 reduced PCA to one instruction: *find the eigenvectors of C and sort them by eigenvalue*. This section says why that instruction is always answerable, and records two facts we use constantly.

5.1 The spectral theorem does the heavy lifting

For a general square matrix, eigenvectors can be complex, can fail to be perpendicular, and can fail to form a basis at all. None of that can happen here, because C is symmetric ($C_{jk} = C_{kj}$ — the covariance of columns j and k does not care about the order).

What symmetry buys us

For a real symmetric $D \times D$ matrix C :

- all D eigenvalues are **real**;
- eigenvectors for different eigenvalues are automatically **perpendicular**;
- there is a full **orthonormal basis** $\mathbf{w}_1, \dots, \mathbf{w}_D$ of eigenvectors, so $C = W\Lambda W^\top$ with $W^\top W = I$ and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_D)$.

And because C is a covariance matrix, every $\lambda_d \geq 0$: by (13) an eigenvalue is a variance, and a variance cannot be negative.

That is why we can always sort $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D \geq 0$ and always take the top M : the list is real, ordered, and complete. It is also why the principal components come out mutually perpendicular without our asking.

5.2 The total variance is the sum of the eigenvalues

This one identity powers every “how much variance do we keep” calculation.

The total variance in the data is the sum of the variances of the individual columns, which is the sum of the diagonal entries of C — its **trace**:

$$\text{total variance} = \sum_{d=1}^D \text{Var}(\text{column } d) = \sum_{d=1}^D C_{dd} = \text{tr}(C). \quad (16)$$

Now use $C = W\Lambda W^\top$ and the fact that the trace does not change when you cycle a product, $\text{tr}(AB) = \text{tr}(BA)$:

$$\text{tr}(C) = \text{tr}(W\Lambda W^\top) = \text{tr}(\Lambda W^\top W) = \text{tr}(\Lambda) = \sum_{d=1}^D \lambda_d. \quad (17)$$

Rotating the axes does not create or destroy spread

$$\underbrace{C_{11} + C_{22} + \cdots + C_{DD}}_{\text{variance in the original columns}} = \underbrace{\lambda_1 + \lambda_2 + \cdots + \lambda_D}_{\text{variance along the components}}$$

PCA does not change the total amount of variation in the data. It only *redistributes* it, pushing as much as possible into the first few directions. That is exactly why keeping the top few components loses so little.

Immediate consequence — the formula you will use over and over:

$$\text{share of variance explained by the top } M = \frac{\lambda_1 + \cdots + \lambda_M}{\lambda_1 + \cdots + \lambda_D} = \frac{\lambda_1 + \cdots + \lambda_M}{\text{tr}(C)}.$$

5.3 Two ways to actually get the eigenvectors

The slides give two routes, and they are genuinely different computations that give the same answer.

	Route A: eigendecomposition	Route B: SVD of the data
what you do	build $C = \frac{1}{N} X^\top X$, then eigendecompose C	decompose $X = U\Sigma V^\top$ directly; never build C
you get	λ_d and w_d straight away	σ_d and v_d ; convert with $\lambda_d = \sigma_d^2 / N$
good because	it is the definition, and it is what you do by hand	more accurate numerically, and it is what most software does
costly when	D is large — C alone is $D \times D$	rarely; you can even stop after the top M vectors

Route A is what you will do on paper, and it is what §15’s questions ask for. Route B is the subject of the next section.

Section recap. C is symmetric, so it has D real, non-negative eigenvalues and a full orthonormal set of eigenvectors. $\text{tr}(C) = \sum_d \lambda_d$, so the eigenvalues are just the total variance re-parcelled among perpendicular directions — which is what makes the “percentage of variance explained” calculation possible.

6 The same picture through the SVD

Any matrix factors as $X = U\Sigma V^\top$ — turn, stretch, turn — and feeding that into C collapses it straight into the eigendecomposition, with $\lambda_d = \sigma_d^2/N$.

The singular value decomposition writes any $N \times D$ matrix as

$$X = U\Sigma V^\top, \quad (18)$$

where U and V have orthonormal columns ($U^\top U = I$, $V^\top V = I$) and Σ is diagonal with the non-negative singular values $\sigma_1 \geq \sigma_2 \geq \dots$ on it.

Turn, stretch, turn

Every matrix, however complicated it looks, does exactly three things in sequence: it *turns* space, it *stretches* the axes by fixed amounts, and it *turns* the result again. That is the whole content of the SVD. $V^\top$ is the first turn, Σ is the stretch, U is the second turn. Follow a unit circle through those three moves and you get an ellipse. The ellipse's axis lengths are the singular values, and its axis directions are the columns of U .

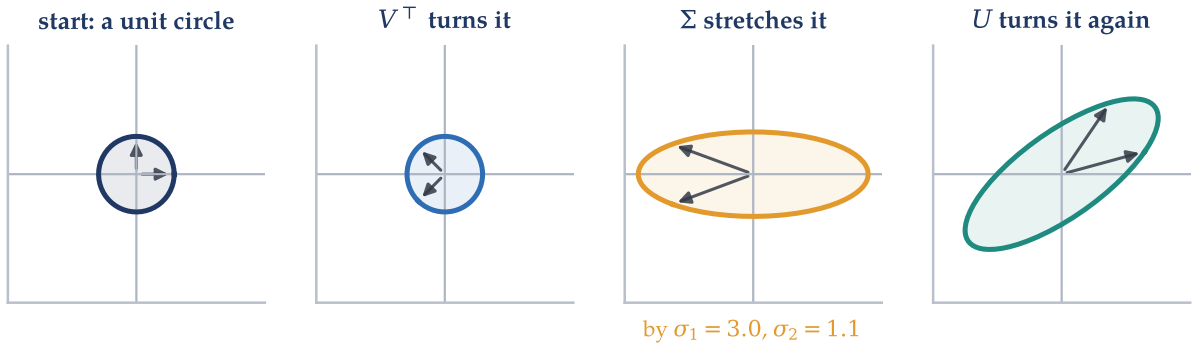


Figure 4: The three moves of a matrix, applied to a unit circle. *How to read it:* nothing is distorted in panels 2 and 4 — they are pure rotations. All the shape change happens in panel 3, and the amounts are exactly σ_1 and σ_2 .

6.1 Substituting the SVD into the covariance matrix

Now watch what happens when the SVD meets C . Start from the definition, substitute $X = U\Sigma V^\top$, and simplify:

$$C = \frac{1}{N} X^\top X = \frac{1}{N} (U\Sigma V^\top)^\top (U\Sigma V^\top) \quad (19)$$

$$= \frac{1}{N} V \Sigma^\top \underbrace{U^\top U}_{=I} \Sigma V^\top \quad U \text{ has orthonormal columns} \quad (20)$$

$$= \frac{1}{N} V \underbrace{\Sigma^\top \Sigma}_{=\Sigma^2} V^\top \quad \Sigma \text{ is diagonal, so } \Sigma^\top = \Sigma \quad (21)$$

$$= V \left(\frac{\Sigma^2}{N} \right) V^\top. \quad (22)$$

Compare the last line with the eigendecomposition $C = W\Lambda W^\top$ from §5. They have the same shape — an orthonormal matrix, a diagonal matrix, and the transpose of the orthonormal

matrix. So they must be the same decomposition:

$$W = V \quad \text{and} \quad \lambda_d = \frac{\sigma_d^2}{N} \quad (23)$$

The principal directions are the right singular vectors of the centred data, and the eigenvalues are the squared singular values divided by N . That is exactly equation (4) of the slides.

Why squared, and why divided by N

Both factors have a plain reading.

Squared because C is built from $X^\top X$ — the data matrix appears *twice*. A stretch of σ in X becomes a stretch of σ^2 in $X^\top X$.

Divided by N because $X^\top X$ is a sum over all N points, while a variance is an *average*. The $1/N$ is the averaging step, and nothing more.

Sanity check: σ_d has the units of the data, so σ_d^2 has squared units — which is right for a variance.

The same result in the slides' notation

The slides put the data points in the *columns*, so X is $D \times N$ and $S = \frac{1}{N} X X^\top$. Substituting $X = U \Sigma V^\top$ there gives

$$S = \frac{1}{N} U \Sigma \underbrace{V^\top V}_{=I} \Sigma^\top U^\top = U \left(\frac{\Sigma \Sigma^\top}{N} \right) U^\top,$$

so the *columns* of U are the eigenvectors of S , and again $\lambda_d = \sigma_d^2 / N$.

Same theorem, different letter, because the data was transposed. If you are ever unsure which of U and V you want, ask which one has D rows — that is the one living in feature space, and that is the one holding the directions.

6.2 Why anyone bothers with route B

If the two routes agree, why does almost every library use the SVD?

- **It never forms C .** Building $X^\top X$ squares the condition number of the problem, which throws away precision. The SVD works on X itself.
- **It links the two views of PCA.** The slides derive PCA twice: once by maximising projected variance, once by minimising reconstruction error. Equation (23) is the bridge — the same U, Σ, V answer both questions. That is why the slides call this “the connection between the maximum variance view and the singular value decomposition”.
- **It hands you low-rank approximation for free**, which is the next section.

Section recap. $X = U \Sigma V^\top$ is turn–stretch–turn. Substituting it into $C = \frac{1}{N} X^\top X$ collapses to $V(\Sigma^2/N)V^\top$, so the right singular vectors *are* the principal directions and $\lambda_d = \sigma_d^2/N$. For the slides' column layout the same argument gives the left singular vectors U instead.

7 Low-rank approximation: the Eckart–Young theorem

Truncating the SVD after M terms gives the best possible rank- M version of the data — which is another way of saying that keeping the top M components loses as little as it is possible to lose.

This section is short. It is not new machinery; it is a second interpretation of what we have already done, and it is worth knowing because it explains the phrase “minimising reconstruction error” on the summary slide.

7.1 The question

Ask for the best simplification of the data at a fixed budget:

$$\tilde{X}_M = \arg \min_{\text{rank}(A) \leq M} \|X - A\|. \quad (24)$$

Among all matrices of rank at most M — that is, all datasets that really live in an M -dimensional subspace — which one sits closest to our actual data?

7.2 The answer

Eckart–Young

Take the SVD and simply stop after the top M singular values:

$$\tilde{X}_M = U_M \Sigma_M V_M^\top,$$

where U_M and V_M hold the first M columns of U and V , and Σ_M holds the M largest singular values. No rank- M matrix gets closer. This holds for both the spectral norm and the Frobenius norm.

The leftover error is made of exactly the pieces you dropped:

$$\|X - \tilde{X}_M\|_2 = \sigma_{M+1}, \quad \|X - \tilde{X}_M\|_F = \sqrt{\sigma_{M+1}^2 + \sigma_{M+2}^2 + \dots} \quad (25)$$

So the error is small precisely when the discarded singular values are small — which, by (23), is precisely when the discarded eigenvalues are small.

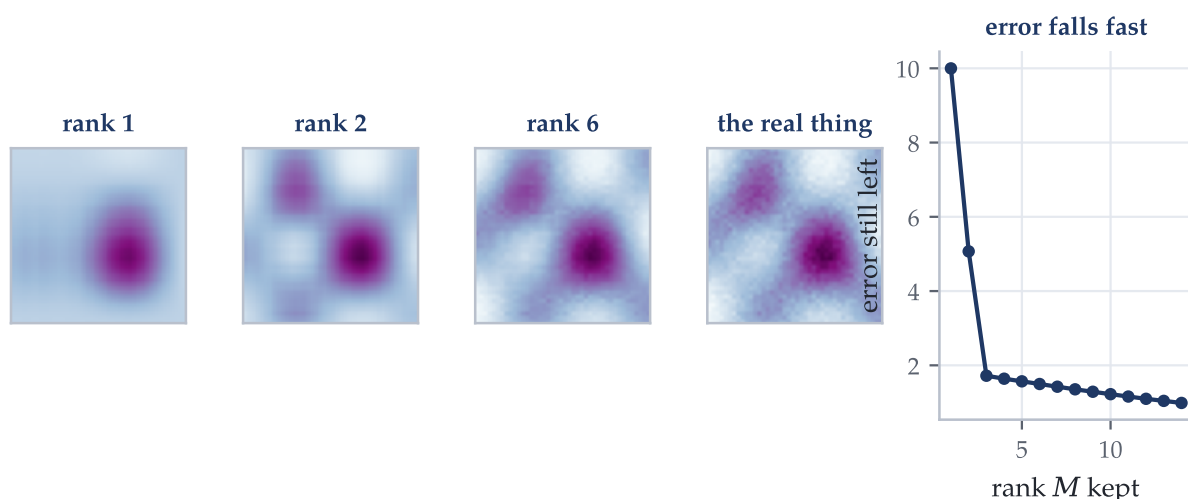


Figure 5: The same matrix rebuilt from its first 1, 2 and 6 singular directions. *How to read it:* rank 1 catches the dominant blob, rank 2 finds the second, rank 6 is already close to the original. The last panel shows the error dropping steeply and then flattening — the shape of almost every real dataset.

A packing analogy

You are packing a suitcase and can take M items. Eckart–Young says the best strategy is exactly what you would guess: pack the biggest, most useful items first, in order, and stop when the case is full. What is not obvious — and is what the theorem actually proves — is that no clever repacking beats this greedy order. Taking the top M singular directions really is optimal, not just sensible.

This is the second derivation of PCA, in disguise

The slides say PCA can be derived two ways: by *maximising the projected variance*, or by *minimising the average reconstruction error*. We did the first in §4. Eckart–Young is the second.

Reconstructing every point from M components gives $\tilde{X} = XWW^\top$, a rank- M matrix. Asking for the smallest possible $\frac{1}{N} \sum_i \|x_i - \tilde{x}_i\|^2$ is exactly problem (24), and its solution is the top M singular directions — the same W we found by maximising variance.

Both routes end at the top eigenvectors of C because of (17): total variance is fixed, so *keeping the most and losing the least* are the same instruction.

Section recap. Truncating the SVD at M terms is the provably best rank- M approximation of the data, with leftover error built from the singular values you dropped. This is the “minimum reconstruction error” view of PCA, and it lands on the same components as the “maximum variance” view.

8 Getting the eigenvectors in practice

The characteristic polynomial works by hand up to 4×4 and then stops working for anyone, so real software iterates — and if you only want the top direction, one very short loop does it.

8.1 The textbook method, and where it breaks

By hand you find eigenvalues by solving the characteristic equation

$$\det(C - \lambda I) = 0. \quad (26)$$

For a 2×2 matrix this is a quadratic and you use the usual formula. For 3×3 it is a cubic, still solvable in radicals. For 4×4 it is a quartic — there is a formula, though nobody enjoys it.

At 5×5 the method dies, and not for want of effort.

Abel–Ruffini: this is not a gap in our cleverness

The characteristic polynomial of a $D \times D$ matrix has degree D . The Abel–Ruffini theorem proves that a general polynomial of degree 5 or more has **no** solution formula in radicals. Not “none is known” — none can exist.

So for $D \geq 5$ there is no closed-form route from a matrix to its eigenvalues. Every practical eigenvalue algorithm is therefore *iterative*: start with a guess and improve it until it stops changing.

This is worth stating clearly because it changes what “solving” means. In an exam, with a 2×2 or 3×3 matrix, the characteristic polynomial is the method. In software, on a 500×500 matrix, nothing of the sort is happening — it is running an iteration.

8.2 Do not compute what you will throw away

There is a second practical point. A full eigendecomposition of a $D \times D$ matrix costs roughly $O(D^3)$ operations and returns all D eigenvectors. But PCA usually wants $M = 2$ or $M = 10$ of them and discards the other several hundred. Paying the full cost and then binning most of the result is wasteful.

Iterative methods that go directly after the top few eigenvectors are substantially cheaper, and they are what libraries call when you ask for a fixed number of components.

8.3 Power iteration: the whole idea in one line

The simplest such method, for the single largest eigenvector, is **power iteration**. Pick a starting vector x_0 that has some component along w_1 — that is, $w_1^\top x_0 \neq 0$, which a randomly chosen vector satisfies with probability 1 — then repeat

$$x_{k+1} = \frac{C x_k}{\|C x_k\|}, \quad k = 0, 1, 2, \dots \quad (27)$$

Multiply by C ; rescale back to unit length; repeat. That is the entire algorithm.

Why it works. Write the starting vector in the eigenbasis — which we may, since the eigenvectors form a complete orthonormal set:

$$x_0 = c_1 w_1 + c_2 w_2 + \dots + c_D w_D. \quad (28)$$

Each multiplication by C scales the w_d part by λ_d , so after k steps (before rescaling)

$$C^k x_0 = c_1 \lambda_1^k w_1 + c_2 \lambda_2^k w_2 + \dots = \lambda_1^k \left[c_1 w_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k w_2 + \dots \right]. \quad (29)$$

Provided the top eigenvalue is strictly the largest, $\lambda_1 > \lambda_2$, every ratio λ_d/λ_1 is less than one, so every term except the first shrinks geometrically towards zero. The w_1 component takes over. The normalisation in (27) just stops the numbers overflowing; it does not affect the direction.

When power iteration crawls

The convergence speed is governed by the ratio λ_2/λ_1 . If the top two eigenvalues are close — say $\lambda_1 = 5.0$ and $\lambda_2 = 4.9$ — the ratio is 0.98 and progress is glacial. In that situation the top two directions carry almost the same variance anyway, so which one comes “first” is not a meaningful question. Real libraries use more sophisticated iterations that handle this case.

Section recap. The characteristic polynomial is a hand method for small matrices; Abel–Ruffini rules it out from 5×5 upwards, so software iterates. When you want only the top few directions, iterating directly is far cheaper than a full decomposition. Power iteration is the minimal example: multiply by C , renormalise, repeat.

9 When D is huge: wide data and the Gram trick

If you have far more columns than rows, never build the $D \times D$ covariance matrix — eigendecompose the $N \times N$ matrix instead and convert the answer back.

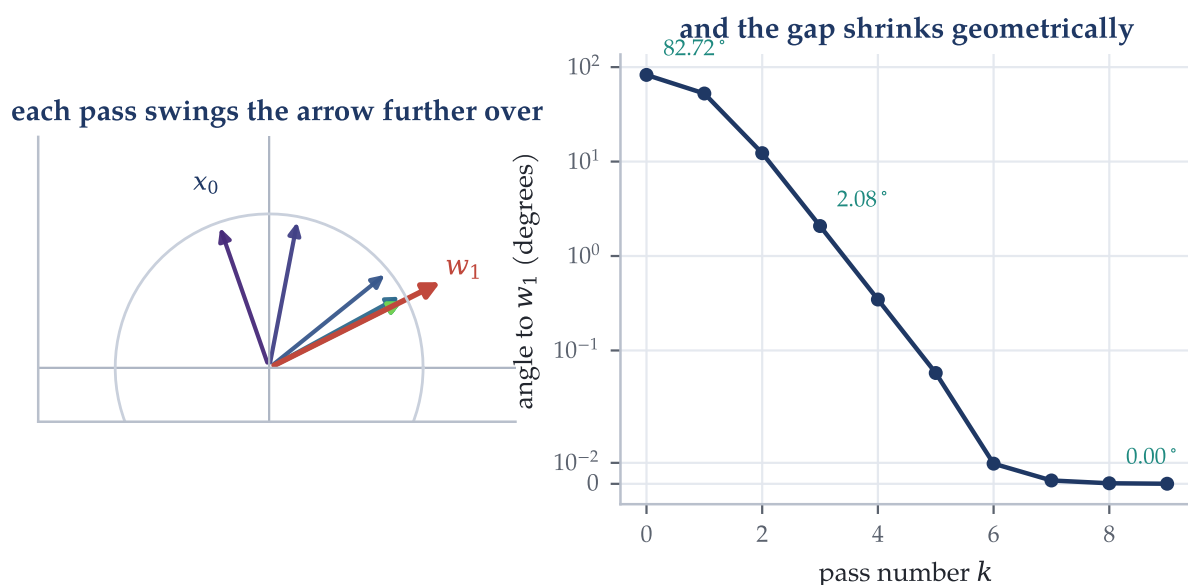


Figure 6: Power iteration on a 2×2 covariance matrix. *How to read it:* left, each pass rotates the arrow further towards the true top eigenvector w_1 (red). Right, the remaining angle on a log scale — a straight-line fall means the error shrinks by a constant factor every pass, here roughly λ_2 / λ_1 .

9.1 Long data and wide data

Most datasets are **long**: many rows, fewer columns. A hundred thousand customers, forty features. Here C is a comfortable 40×40 and everything so far applies directly.

Some datasets are **wide**: far more columns than rows. Fifty tissue samples against twenty thousand genes. Sixty documents against a hundred thousand vocabulary terms. Here $N \ll D$, and the standard recipe falls over.

9.2 Why the standard recipe fails

Take X of size $50 \times 20,000$. Then $X^\top X$ is $20,000 \times 20,000$: four hundred million numbers, about 3.2 GB in double precision — for a dataset whose raw form is only 8 MB. And storage is the smaller problem. Eigendecomposition costs about D^3 operations, which here is 8×10^{12} — hours of computing, to extract directions from fifty data points.

The tempting wrong fix

The natural reaction is: “ $X^\top X$ is huge, so let us use $XX^\top$ instead — it is only 50×50 .” That is the right *instinct* and the wrong *statement*. $XX^\top$ is not the covariance matrix; its rows and columns are indexed by data points, not by features, and its eigenvectors live in the wrong space entirely — they have N components where we need D . The fix below rescues the instinct: use $XX^\top$ as a stepping stone, then convert its eigenvectors into the ones you actually want.

9.3 The trick: one multiplication by X

Start from the eigen-equation we are trying to solve, with $C = \frac{1}{N} X^\top X$:

$$\frac{1}{N} X^\top X w = \lambda w. \quad (30)$$

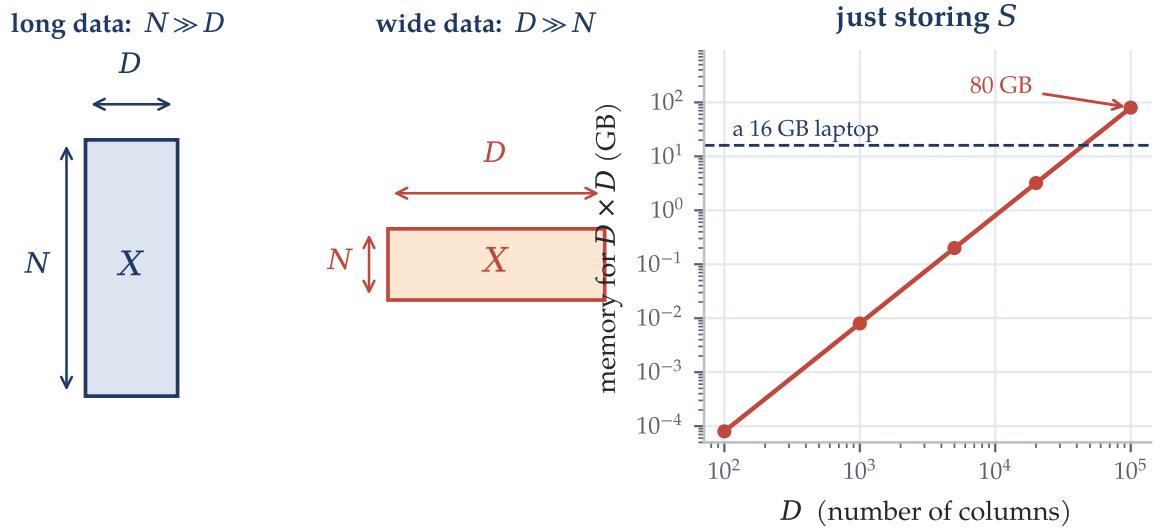


Figure 7: Long against wide, and what wide costs you. *How to read it:* the covariance matrix is $D \times D$ regardless of how few rows you have, so its size is set by the *wide* dimension. Past about $D = 45,000$ it will not fit in a typical laptop at all.

Multiply both sides on the left by X — legal, since it is the same operation on both sides:

$$\frac{1}{N} X X^T (Xw) = \lambda (Xw). \quad (31)$$

Now name the two pieces. Let $K = \frac{1}{N} X X^T$, which is $N \times N$ — the **Gram matrix**. Let $u = Xw$, which has N components. Then

$$\boxed{Ku = \lambda u} \quad (32)$$

which is an eigenvalue problem on a matrix of size $N \times N$: 50×50 in our example, not $20,000 \times 20,000$. And crucially the *eigenvalue* λ is *unchanged* — it is the same λ in both equations.

9.4 Converting back

Solving the small problem gives u ; we need w . Multiply u by X^T and check what happens:

$$C(X^T u) = \frac{1}{N} X^T X X^T u = X^T \left(\frac{1}{N} X X^T \right) u = X^T K u = X^T (\lambda u) = \lambda (X^T u). \quad (33)$$

So $X^T u$ is an eigenvector of C with the very eigenvalue we wanted.

Do not forget to normalise

$X^T u$ points the right way but has the wrong length — the derivation only ever shows proportionality. Since the entire PCA argument assumed unit directions, finish with

$$w = \frac{X^T u}{\|X^T u\|}.$$

Dropping this step is the most common error in this calculation, and it quietly breaks $\text{Var} = \lambda$.

9.5 The recipe, and its limit

Step 1. Centre X (size $N \times D$, with $D \gg N$).

Step 2. Build the Gram matrix $K = \frac{1}{N}XX^\top$, size $N \times N$.

Step 3. Eigendecompose K : get $\lambda_1 \geq \dots \geq \lambda_N$ and vectors $\mathbf{u}_1, \dots, \mathbf{u}_N$.

Step 4. Convert and normalise: $\mathbf{w}_m = X^\top \mathbf{u}_m / \|X^\top \mathbf{u}_m\|$.

Step 5. Project as usual: $Z = XW$.

You could not have had more than N components anyway

A $D \times D$ covariance matrix has D eigenvalues — twenty thousand of them in our example. But $\text{rank}(X) \leq \min(N, D) = N$, and centring removes one more degree of freedom, so

$$\text{rank}(C) \leq N - 1.$$

At most $N - 1$ eigenvalues can be non-zero; all the rest are exactly zero and carry no variance whatsoever.

Fifty data points can only ever define a 49-dimensional cloud, no matter how many columns you measured. So the Gram trick is not an approximation that sacrifices something — the small matrix holds every non-zero eigenvalue there was. You lose nothing at all.

The same trick in the slides' notation

With data in columns (X of size $D \times N$, $S = \frac{1}{N}XX^\top$), start from $S\mathbf{b}_m = \lambda_m\mathbf{b}_m$ and multiply on the left by $X^\top$:

$$\frac{1}{N}X^\top X(X^\top \mathbf{b}_m) = \lambda_m(X^\top \mathbf{b}_m), \quad \mathbf{c}_m := X^\top \mathbf{b}_m,$$

so $\mathbf{c}_m$ is an eigenvector of the small $N \times N$ matrix $\frac{1}{N}X^\top X$. Going the other way, multiply by X to recover $\frac{1}{N}XX^\top(X\mathbf{c}_m) = \lambda_m(X\mathbf{c}_m)$, so $X\mathbf{c}_m$ is an eigenvector of S — normalise it. Those are slide equations (7)–(10), with the roles of X and $X^\top$ swapped because the data is transposed.

Section recap. For wide data, eigendecompose the $N \times N$ Gram matrix $\frac{1}{N}XX^\top$ instead of the $D \times D$ covariance matrix. The eigenvalues are identical; recover the directions with $\mathbf{w} \propto X^\top \mathbf{u}$ and normalise. Nothing is lost, because C never had more than $N - 1$ non-zero eigenvalues to begin with.

10 How many components to keep, and how to read them

Divide each eigenvalue by their total to get its share of the variance; keep enough components to reach the share you need; then read the loadings to find out what those components are made of.

10.1 The share of variance explained

By (17) the eigenvalues add up to the total variance, so each one's share is immediate:

$$\text{share of component } m = \frac{\lambda_m}{\sum_{d=1}^D \lambda_d}, \quad \text{share of the top } M = \frac{\sum_{m=1}^M \lambda_m}{\sum_{d=1}^D \lambda_d}. \quad (34)$$

Plot the individual shares against the component number and you get a **scree plot**; plot the running total and you get the curve people actually make decisions from.

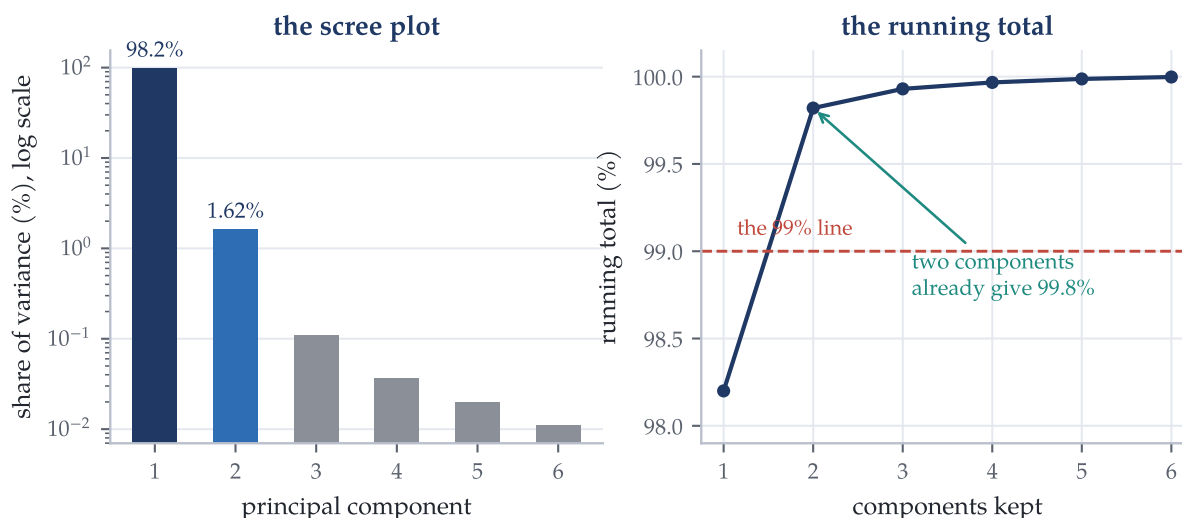


Figure 8: A real example — the diagnostic dataset shown in class. *How to read it:* left, the first component alone carries 98.2% of the variance and the second carries 1.6%; note the log scale, without which bars three onwards would be invisible. Right, two components already reach 99.8%, so components three onwards are not worth carrying.

Three ways people choose M , in rough order of how often they are used:

- **Hit a target share.** Keep adding components until the running total passes 90%, 95% or 99%. Simple, defensible, and the most common choice in practice.
- **Find the elbow.** Look for the point where the scree plot stops falling steeply and flattens out. Everything after the elbow is small and roughly equal — usually noise.
- **Keep $\lambda > 1$ (the Kaiser rule).** Only meaningful when you standardised the columns, because then each original column contributes exactly 1 to the total variance. A component with $\lambda < 1$ is carrying less than a single original column, so it is not paying its way.

A high percentage is not automatically a good sign

If one component explains 98% of the variance, that can mean the data really does lie almost on a line — or it can mean one column is measured in units thousands of times larger than the rest and is simply dominating the total. Check that you standardised (§11) before celebrating.

10.2 The components come out uncorrelated — here is the proof

We claimed this back in §2. It follows in two lines.

The scores are $Z = XW$, and they have mean zero because X does. So their covariance matrix is

$$\text{Cov}(Z) = \frac{1}{N} Z^T Z = \frac{1}{N} (XW)^T (XW) = W^T \left(\frac{X^T X}{N} \right) W = W^T C W. \quad (35)$$

Now use $CW = W\Lambda$ (that is just the eigen-equation, written for all the columns at once) and $W^T W = I$:

$$\text{Cov}(Z) = W^T W \Lambda = \Lambda = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_M \end{pmatrix}. \quad (36)$$

Every off-diagonal entry is zero. The off-diagonal entries *are* the covariances between different components, so the components are mutually uncorrelated. And the diagonal entries are the eigenvalues, which is $\text{Var} = \lambda$ once more, now for all components at once.

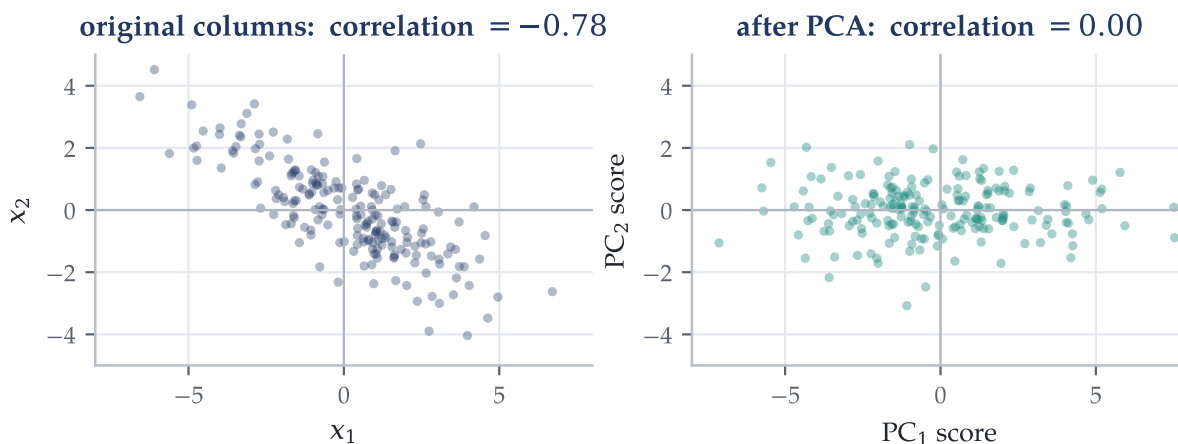


Figure 9: Correlation removed. *How to read it:* left, the two original columns move together, so knowing one tells you a lot about the other — that shared information is redundant. Right, the same points described by their two component scores: the cloud is now axis-aligned and the correlation is zero.

Why this matters downstream

Correlated inputs cause real trouble: unstable regression coefficients, multicollinearity, models that cannot tell which of two twin features is doing the work. The usual fix is to delete one of each correlated pair, which throws information away. PCA solves it differently. It keeps all the information but re-expresses it in perpendicular directions, so the redundancy is gone by construction. You can hand $PC_1, \dots, PC_M$ straight to a model without worrying about correlation between them at all.

10.3 Reading a component: factor loadings

A component is a direction, and its entries are the weights of a recipe:

$$z_1 = \underbrace{a_1}_{\text{loading}} x_1 + a_2 x_2 + \dots + a_D x_D, \quad \mathbf{w}_1 = (a_1, \dots, a_D)^\top. \quad (37)$$

These weights are called the **factor loadings**. They are literally the components of the eigenvector, and they tell you what the component is made of.

Suppose three features — age, experience, certifications — give a first component with loadings $\frac{1}{7}(6, 3, 2) \approx (0.86, 0.43, 0.29)$. Age dominates, so PC_1 is “mostly an age axis”. If the second component has loadings $\frac{1}{\sqrt{5}}(-1, 2, 0) \approx (-0.45, 0.89, 0)$, then PC_2 is “mostly an experience axis”. You have not lost the meaning of your data; you have to look one level deeper for it.

Both vectors have length 1, and their dot product is $\frac{1}{7\sqrt{5}}(-6 + 6 + 0) = 0$ — as any pair of loadings from a real PCA must. If a set of “loadings” fails those two checks, it is not a set of principal components.

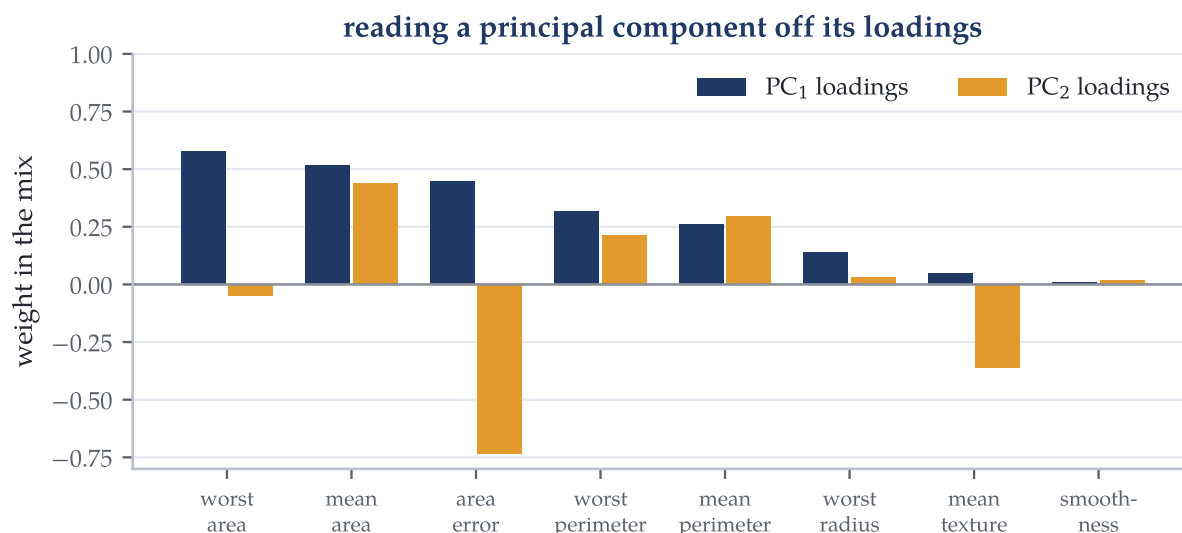


Figure 10: Loadings for the first two components of the diagnostic dataset. *How to read it:* tall bars are the features driving that component. PC₁ is dominated by the area measurements, so it is essentially a “size” axis. Negative bars mean the feature pushes the score the other way.

People say PCA is a black box. It is less black than that

A common complaint is that once you replace your features with components you can no longer say anything to the business. That is overstated. Look at the loadings and you can usually say, in plain language, what each component is measuring — “PC₁ is basically overall tumour size”, “PC₂ contrasts mean area against area error”.

There is a real trade-off, but it is not “interpretable versus uninterpretable”. It is: a model on your original features is easy to explain and may be less accurate; a model on engineered or component features is often more accurate and takes one extra step to explain. Which you want depends on whether the client needs to *understand* the drivers or to *predict* well.

Section recap. $\lambda_m / \sum_d \lambda_d$ is the share of variance a component carries; keep enough to hit your target, or stop at the elbow. The scores are uncorrelated because $\text{Cov}(Z) = W^T C W = \Lambda$ is diagonal. To interpret a component, read its loadings — they are the weights of the linear mix it represents.

11 The recipe: four steps, end to end

Centre, standardise, eigendecompose, project — and if you need the answer in the original units, undo the first two steps at the end.

Everything so far assembles into four steps. This is the procedure the slides lay out, and it is the procedure to follow in an exam.

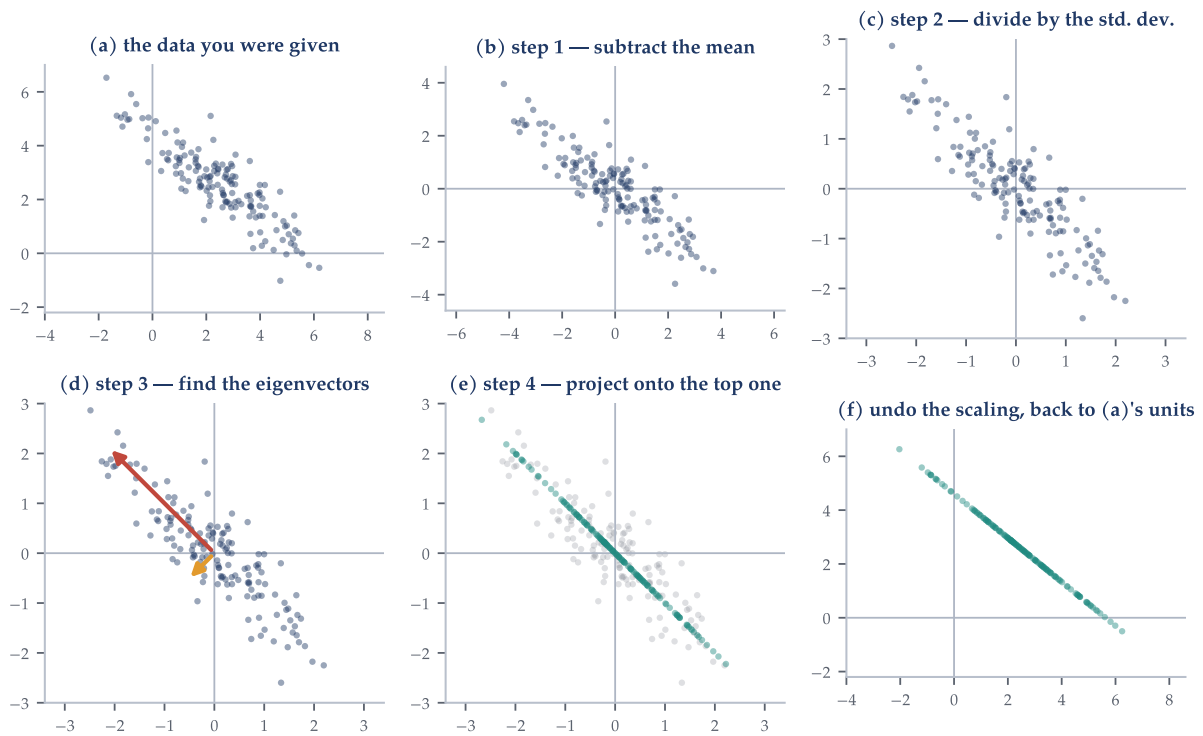


Figure 11: The four steps on a real cloud of points. *How to read it:* (a) to (b) shifts the cloud to the origin; (b) to (c) rescales each axis to unit spread; (d) draws the two eigenvectors, each with length proportional to the square root of its eigenvalue; (e) flattens every point onto the top one; (f) reverses the scaling so the answer is back in the original units.

11.1 Step 1 — Subtract the mean

Compute the mean of each column and subtract it from every entry in that column:

$$x_{id}^{(\text{centred})} = x_{id} - \mu_d, \quad \mu_d = \frac{1}{N} \sum_{i=1}^N x_{id}. \quad (38)$$

Now every column averages to zero, and the cloud is centred on the origin.

Strictly, centring is not *required* — you could work with the uncentred second-moment matrix — but every formula in this document assumes it, including $\text{Var}(z) = \frac{1}{N} \sum z_i^2$ in (8). It also keeps the arithmetic small and well conditioned. Always centre.

11.2 Step 2 — Divide by the standard deviation

Divide each centred column by that column's standard deviation:

$$x_{id}^{(\text{standardised})} = \frac{x_{id} - \mu_d}{\sigma_d}. \quad (39)$$

Every column now has variance 1 and no units at all.

Skip this and the biggest unit wins

PCA maximises variance, and variance depends on units. Measure a length in millimetres instead of metres and its variance grows by a factor of a million; that column will then dominate the first component for no reason other than the choice of unit.

Figure 12 shows this happening: with raw kilograms and metres the first component is essentially “weight”, because weight’s numbers are simply bigger — its loading on weight is 1.00 and on height 0.01. After standardising, the two features compete on equal terms and the component reflects the real relationship.

The exception: if all your columns are already in the same unit and the differences in spread are real information you want PCA to see, standardising would throw that away. Then centre only. Decide deliberately — do not let it happen by accident.

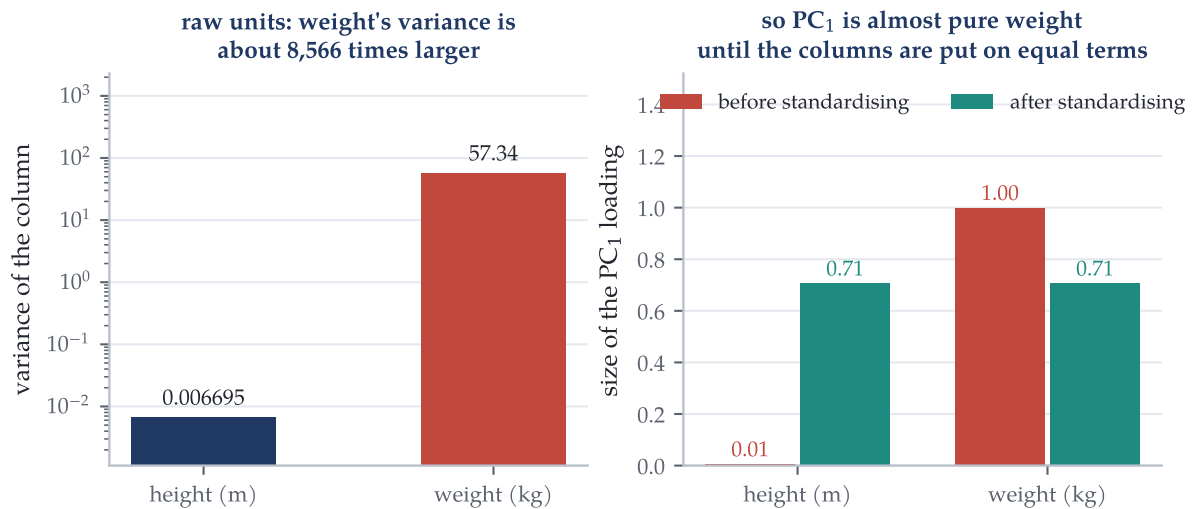


Figure 12: What raw units do to PCA, for height in metres and weight in kilograms. *How to read it:* left, weight’s variance is over eight thousand times larger — not because weight matters more, but because kilograms make bigger numbers than metres. Right, the consequence: in raw units PC₁ puts a loading of 1.00 on weight and 0.01 on height, so it is effectively just the weight column. After standardising, the two share the direction equally.

11.3 Step 3 — Eigendecompose the covariance matrix

Build $C = \frac{1}{N}X^T X$ from the prepared data, then find its eigenvalues and eigenvectors. By the spectral theorem (§5) the eigenvectors form an orthonormal basis. Sort by eigenvalue, largest first, and collect the top M eigenvectors as the columns of W :

$$W = [w_1 \mid w_2 \mid \cdots \mid w_M] \in \mathbb{R}^{D \times M}. \quad (40)$$

Normalise the eigenvectors

Solving $(C - \lambda I)w = \mathbf{0}$ by hand gives a *direction*, and any multiple of it satisfies the equation. You must scale it to unit length before using it:

$$w \leftarrow \frac{w}{\|w\|}.$$

If you skip this, the projected variance will not equal λ , $W^T W$ will not be I , and the reconstruction $WW^T x$ will be wrong. It is one line, and it is the line most often dropped.

11.4 Step 4 — Project

Multiply the prepared data by W :

$$Z = XW \quad (\text{size } N \times M). \quad (41)$$

Row i of Z holds the M coordinates of point i in the new basis. You began with D columns and now have M . That is the dimension reduction, and it is a single matrix multiplication.

Projecting a point you have never seen. For a new point x_* — a test row, a fresh customer — the mechanics are the same, with one condition attached:

$$x_*^{(d)} \longleftarrow \frac{x_*^{(d)} - \mu_d}{\sigma_d}, \quad z_* = W^\top x_*, \quad \tilde{x}_* = Wz_* = WW^\top x_*. \quad (42)$$

Use the *training* mean and standard deviation

The μ_d and σ_d above are the ones computed from the training data. Not from the test set, and not recomputed on the single new point — which would be meaningless anyway, since one point has no spread.

The same applies to W . Mean, standard deviation and components are all fitted once, on training data, and then applied unchanged to anything new. Recomputing them on test data leaks information and inflates your reported accuracy.

11.5 Step 5 (optional) — Undo the preparation

$\tilde{x}_*$ is still standardised. To read it in the original units, reverse steps 2 and 1, in that order:

$$x_{\text{original units}}^{(d)} = \tilde{x}_*^{(d)} \cdot \sigma_d + \mu_d. \quad (43)$$

That is panel (f) of Figure 11. The points now lie on a line in the original coordinate system — the same data, compressed to M dimensions and unpacked again.

Why $WW^\top$ is not the identity, and why that is the point

$W^\top W = I_M$ always — the columns are orthonormal, so that product is a small $M \times M$ identity.

But $WW^\top$ is $D \times D$ and equals I_D *only* when $M = D$. When $M < D$ it is a *projector*: it flattens any vector onto the M -dimensional slice that W spans, and everything perpendicular to that slice is destroyed. That lost perpendicular part is exactly the reconstruction error, and its mean squared length, $\frac{1}{N} \sum_i \|x_i - \tilde{x}_i\|^2$, equals $\lambda_{M+1} + \dots + \lambda_D$ — the eigenvalues you chose not to keep.

Order matters, and it is a useful check: $W^\top W$ small and equal to I ; $WW^\top$ big and not equal to I .

Section recap. Centre; standardise; eigendecompose C and keep the top M unit eigenvectors as W ; project with $Z = XW$. For new points, reuse the training μ, σ and W . To return to original units, undo the standardising and add the mean back.

12 A worked example, start to finish

Five points, two columns, every number exact — centre, covariance, characteristic polynomial, eigenvectors, projection, and three checks that confirm the answer.

Here is the whole procedure on numbers small enough to verify by hand. This is the shape of the calculation a question paper asks for.

The data

Five points in two dimensions:

$$(1,2), \quad (4,5), \quad (7,8), \quad (8,4), \quad (10,6).$$

Reduce this dataset from two dimensions to one.

Step 1 — Centre the data. Column means:

$$\mu_1 = \frac{1+4+7+8+10}{5} = \frac{30}{5} = 6, \quad \mu_2 = \frac{2+5+8+4+6}{5} = \frac{25}{5} = 5. \quad (44)$$

Subtract (6,5) from every point:

$$X = \begin{pmatrix} -5 & -3 \\ -2 & 0 \\ 1 & 3 \\ 2 & -1 \\ 4 & 1 \end{pmatrix} \quad (45)$$

Check: each column of X must sum to zero. $-5 - 2 + 1 + 2 + 4 = 0$ and $-3 + 0 + 3 - 1 + 1 = 0$. Good.

We skip standardisation here: both columns are in the same units and have comparable spread, so there is nothing to correct for.

Step 2 — Build the covariance matrix.

$$\sum x_1^2 = 25 + 4 + 1 + 4 + 16 = 50, \quad C_{11} = 50/5 = 10, \quad (46)$$

$$\sum x_2^2 = 9 + 0 + 9 + 1 + 1 = 20, \quad C_{22} = 20/5 = 4, \quad (47)$$

$$\sum x_1 x_2 = 15 + 0 + 3 - 2 + 4 = 20, \quad C_{12} = 20/5 = 4. \quad (48)$$

$$C = \frac{1}{N} X^T X = \begin{pmatrix} 10 & 4 \\ 4 & 4 \end{pmatrix} \quad (49)$$

Divide by $N = 5$, not by 4. Total variance = $\text{tr}(C) = 10 + 4 = 14$.

Step 3 — Eigenvalues from the characteristic polynomial.

$$\det(C - \lambda I) = \begin{vmatrix} 10 - \lambda & 4 \\ 4 & 4 - \lambda \end{vmatrix} = (10 - \lambda)(4 - \lambda) - 16 = \lambda^2 - 14\lambda + 24 = 0. \quad (50)$$

$$\lambda = \frac{14 \pm \sqrt{196 - 96}}{2} = \frac{14 \pm 10}{2} \implies \lambda_1 = 12, \quad \lambda_2 = 2. \quad (51)$$

Two checks, both instant: $\lambda_1 + \lambda_2 = 14 = \text{tr}(C)$, and $\lambda_1 \lambda_2 = 24 = \det(C)$. Both hold.

Shares of the variance:

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{12}{14} = 85.71\%, \quad \frac{\lambda_2}{\lambda_1 + \lambda_2} = \frac{2}{14} = 14.29\%. \quad (52)$$

One component keeps almost 86% of the variation, so reducing to one dimension is reasonable.

Step 4 — Eigenvectors, then normalise. For $\lambda_1 = 12$, solve $(C - 12I)w = 0$:

$$\begin{pmatrix} -2 & 4 \\ 4 & -8 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies -2a + 4b = 0 \implies a = 2b. \quad (53)$$

Taking $b = 1$ gives the direction $(2, 1)$, whose length is $\sqrt{4+1} = \sqrt{5}$:

$$w_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix} \approx \begin{pmatrix} 0.8944 \\ 0.4472 \end{pmatrix}. \quad (54)$$

For $\lambda_2 = 2$, solve $(C - 2I)w = 0$: $8a + 4b = 0$, so $b = -2a$, giving $(1, -2)$ and

$$w_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 1 \\ -2 \end{pmatrix}. \quad (55)$$

Check: $w_1^\top w_2 = (2 \cdot 1 + 1 \cdot (-2))/5 = 0$. Perpendicular, as the spectral theorem promised. And each has length 1.

Step 5 — Project onto the first component. With $W = w_1$, compute $Z = XW$, one dot product per row:

$$\begin{aligned} (-5, -3) : \quad & \frac{1}{\sqrt{5}}(2(-5) + 1(-3)) = -\frac{13}{\sqrt{5}} \approx -5.814 \\ (-2, 0) : \quad & \frac{1}{\sqrt{5}}(2(-2) + 1(0)) = -\frac{4}{\sqrt{5}} \approx -1.789 \\ (1, 3) : \quad & \frac{1}{\sqrt{5}}(2(1) + 1(3)) = \frac{5}{\sqrt{5}} \approx 2.236 \\ (2, -1) : \quad & \frac{1}{\sqrt{5}}(2(2) + 1(-1)) = \frac{3}{\sqrt{5}} \approx 1.342 \\ (4, 1) : \quad & \frac{1}{\sqrt{5}}(2(4) + 1(1)) = \frac{9}{\sqrt{5}} \approx 4.025 \end{aligned} \quad (56)$$

Five points, five numbers. The reduction is done.

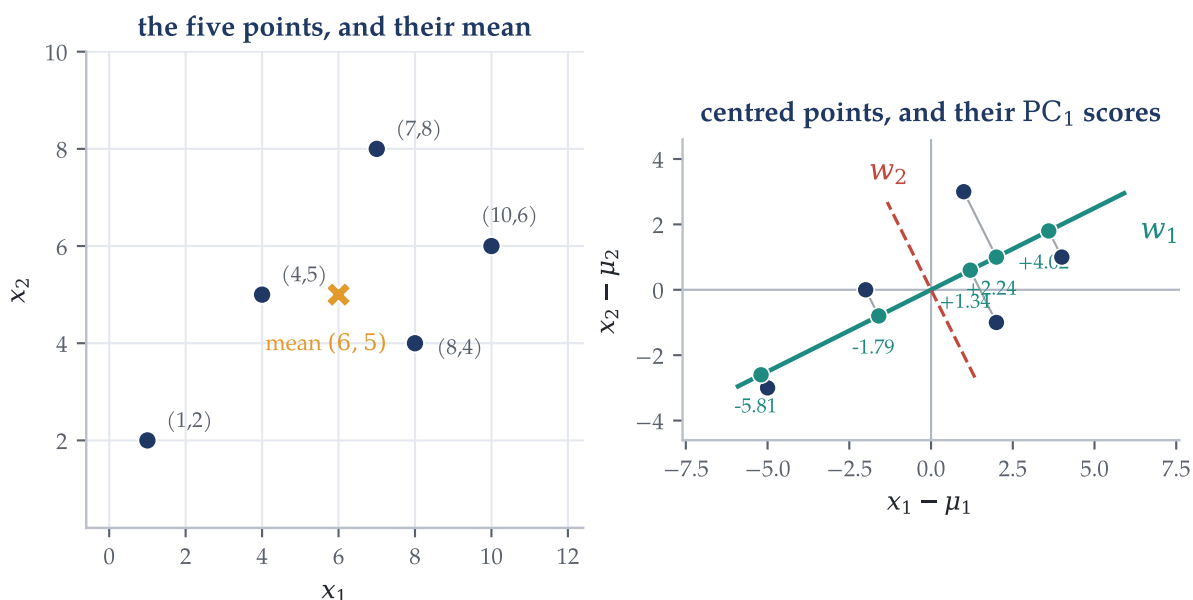


Figure 13: The worked example. *How to read it:* left, the five raw points with their mean marked. Right, the centred points dropped onto w_1 (solid) with their scores written underneath; w_2 (dashed) is the direction we discard.

Step 6 — Three checks.

Step 1. The scores average to zero. $(-13 - 4 + 5 + 3 + 9)/\sqrt{5} = 0/\sqrt{5} = 0$. As they must, since X was centred.

Step 2. The variance of the scores equals λ_1 .

$$\frac{1}{5} \cdot \frac{169 + 16 + 25 + 9 + 81}{5} = \frac{300}{25} = 12 = \lambda_1. \checkmark$$

This is (13) confirmed on real numbers.

Step 3. The discarded direction accounts for the rest. Projecting onto w_2 gives scores $\frac{1}{\sqrt{5}}(1, -2, -5, 4, 2)$, whose variance is $\frac{1}{5} \cdot \frac{1+4+25+16+4}{5} = \frac{50}{25} = 2 = \lambda_2$, and $12 + 2 = 14 = \text{tr}(C)$. Nothing has gone missing.

Step 7 — A new point. A new observation arrives: $x_* = (12, 9)$. Centre it with the *training* mean: $(12 - 6, 9 - 5) = (6, 4)$. Then

$$z_* = w_1^\top \begin{pmatrix} 6 \\ 4 \end{pmatrix} = \frac{2(6) + 1(4)}{\sqrt{5}} = \frac{16}{\sqrt{5}} \approx 7.155. \quad (57)$$

Rebuild it from that single number:

$$\tilde{x}_*^{(\text{centred})} = z_* w_1 = \frac{16}{\sqrt{5}} \cdot \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 6.4 \\ 3.2 \end{pmatrix}, \quad \tilde{x}_* = \begin{pmatrix} 6.4 \\ 3.2 \end{pmatrix} + \begin{pmatrix} 6 \\ 5 \end{pmatrix} = \begin{pmatrix} 12.4 \\ 8.2 \end{pmatrix}. \quad (58)$$

The reconstruction error is the part that fell off:

$$\left\| \begin{pmatrix} 6 \\ 4 \end{pmatrix} - \begin{pmatrix} 6.4 \\ 3.2 \end{pmatrix} \right\|^2 = (-0.4)^2 + (0.8)^2 = 0.80, \quad (59)$$

which is exactly z_2^2 , where $z_2 = w_2^\top (6, 4)^\top = (6 - 8)/\sqrt{5}$ gives $z_2^2 = 4/5 = 0.80$. The error really is the squared score along the direction we threw away.

A question may hand you the raw data, or it may hand you C directly to save you the arithmetic. If you are given C , start at Step 3 — the rest is identical. Always state which λ is largest, always normalise the eigenvectors, and if you have thirty seconds spare, check $\sum \lambda = \text{tr}(C)$ and $\prod \lambda = \det(C)$.

13 Traps, limits, and honest caveats

PCA is linear, unit-sensitive, and blind to your target — knowing when those three things bite is most of what separates using it well from using it by habit.

13.1 PCA is linear, and only linear

Every component is a straight-line combination $z = a_1 x_1 + \dots + a_D x_D$, so PCA can only find flat structure. If your data lies on a curve, a spiral or a sphere, no straight direction describes it well and PCA will report that several components are needed when really there is one curved one.

There are non-linear alternatives — kernel PCA, and manifold methods such as t-SNE and UMAP. What matters here is knowing the boundary: PCA's honest claim is about *linear* structure.

13.2 PCA is blind to your target

This is the deepest limitation, and it is the flip side of §2's most useful property. PCA ranks directions by variance in x . It has no idea whether those directions have anything to do with y , and Figure 14 shows a case where it ranks them exactly backwards.

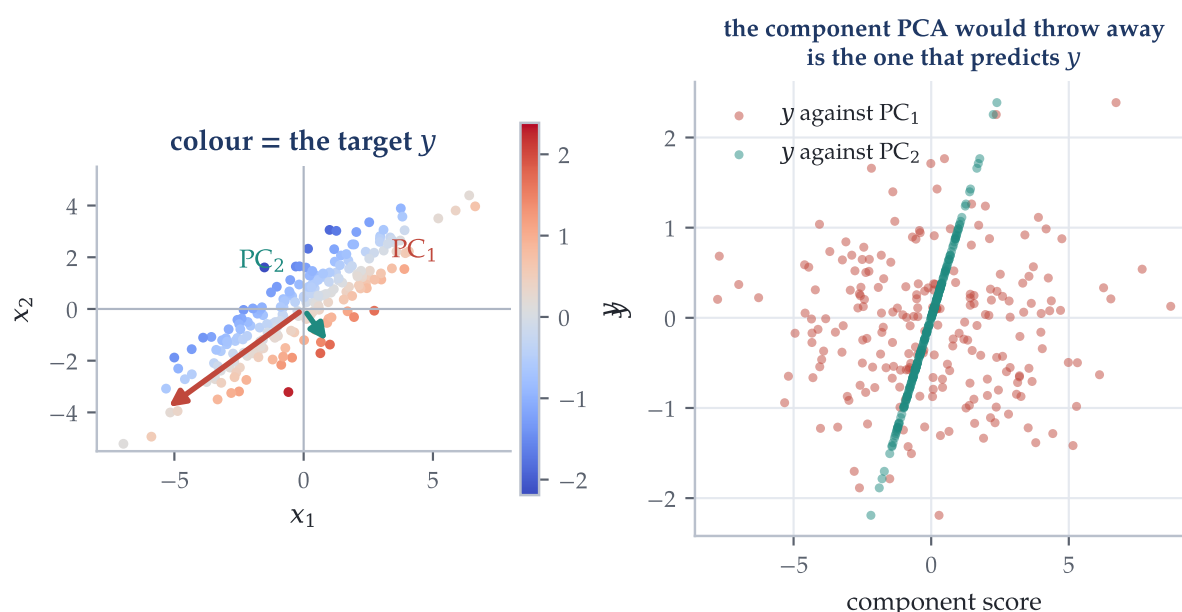


Figure 14: A case where PCA gets it exactly wrong. *How to read it:* colour is the target y , and it changes across the *narrow* direction of the cloud. On the right, y plotted against each component: PC_1 — the one PCA ranks first — has no relationship with y at all, while PC_2 , the one you would drop, predicts it perfectly.

Figure 14 is not a contrived picture; this happens whenever the signal you care about is subtle and the dominant variation is something else — lighting in images, batch effects in gene data, overall loudness in audio. If you need a reduction that keeps target information, use a supervised method (partial least squares, linear discriminant analysis) instead of, or alongside, PCA.

13.3 Units decide the answer

Covered in §11 and shown in Figure 12: because variance carries units, changing a column's unit changes the components. Always make a deliberate decision about standardising.

13.4 Outliers pull the components around

Variance is a sum of *squared* deviations, so a single far-away point contributes enormously. One mis-keyed value can swing the first component towards itself. Look at the data before running PCA, and treat a component that seems to be describing one point as a warning sign.

13.5 The sign of a component is arbitrary

If w satisfies $Cw = \lambda w$ then so does $-w$, with the same eigenvalue. Both are equally valid answers, and different software will hand you different signs for the same dataset. Nothing is wrong — the scores simply flip sign with the vector, and every variance is unchanged. Do not read meaning into whether a loading is $+0.7$ or -0.7 on its own; read the *pattern* of signs within a component.

13.6 Components are new features, not old ones

Dropping components is not the same as dropping columns. After PCA you are no longer measuring age or experience — you are measuring mixtures. Every original column still contributes to every component. If your client needs a model stated in terms of the things they actually measured, PCA gives you accuracy at the cost of one extra explanatory step (\$10's loadings).

Section recap. PCA finds linear structure only; it ranks by variance in x and can discard exactly the direction that predicts y ; it is sensitive to units and to outliers; component signs are arbitrary; and its outputs are mixtures, not original columns.

14 The one-page card

Everything above, on one page.

THE PROBLEM

Find the unit direction along which the projected data spreads out the most:

$$\max_w w^\top C w \quad \text{subject to} \quad w^\top w = 1.$$

A constrained optimisation problem: the constraint is there because only the *direction* matters, not the length.

THE DERIVATION, IN SIX LINES

1. $z = Xw$ (project)
2. $\text{Var}(z) = \frac{1}{N} z^\top z = \frac{1}{N} (Xw)^\top (Xw) = w^\top C w$
3. $\mathcal{L} = w^\top C w - \lambda(w^\top w - 1)$ (Lagrangian)
4. $\partial \mathcal{L} / \partial w = 2Cw - 2\lambda w = 0$
5. $Cw = \lambda w$ (so w must be an eigenvector)
6. $w^\top C w = w^\top \lambda w = \lambda$ (so the maximum variance is λ)

THE FIVE FACTS

$Cw_d = \lambda_d w_d$	directions are eigenvectors of C
$\text{Var}(\text{score}_d) = \lambda_d$	eigenvalue = variance along it
$\text{tr}(C) = \sum_d \lambda_d$	PCA moves variance, never creates it
$\text{Cov}(Z) = W^\top C W = \Lambda$	components are uncorrelated
$\lambda_d = \sigma_d^2 / N$	the bridge from the SVD of X

THE RECIPE

1. **Centre.** Subtract each column's mean.
2. **Standardise.** Divide each column by its standard deviation, unless all columns share a unit.
3. **Eigendecompose.** $C = \frac{1}{N} X^\top X$. Solve $\det(C - \lambda I) = 0$, then $(C - \lambda I)w = 0$; **normalise each** w ; sort by λ .
4. **Choose M .** Keep enough components for $(\sum_{m \leq M} \lambda_m) / \text{tr}(C)$ to reach your target.
5. **Project.** $Z = XW$. For a new point: standardise with the *training* μ, σ , then $z_* = W^\top x_*$, $\tilde{x}_* = WW^\top x_*$.

THE SIX TRAPS

1. Dividing by $N - 1$. This course divides by N .
2. Forgetting to normalise the eigenvectors.
3. Forgetting to centre before building C .
4. Standardising the test set with its own μ, σ instead of the training ones.
5. Expecting $WW^\top = I$. Only $W^\top W = I$; $WW^\top$ is a projector.
6. Assuming the top component predicts y . PCA never looked at y .

WHEN THINGS GET BIG

$D \geq 5$, by hand	no formula exists (Abel–Ruffini); software iterates
only the top one	power iteration $x_{k+1} = Cx_k / \ Cx_k\ $
$D \gg N$, wide data	eigendecompose $K = \frac{1}{N} XX^\top$ ($N \times N$); same λ ; then $w = X^\top u / \ X^\top u\ $; at most $N - 1$ are non-zero
best rank- M picture	Eckart–Young: $\tilde{X}_M = U_M \Sigma_M V_M^\top$
three quick checks	$\sum_d \lambda_d = \text{tr}(C)$ • $\prod_d \lambda_d = \det(C)$ • $w_1^\top w_2 = 0$ and $\ w_d\ = 1$

15 Practice set: seven questions, fully solved

Seven different question types, each with a note on how to attack that type, then a complete solution. Try each one before reading on.

Notation for this section. These questions follow the convention used in question papers rather than the one used in the body of this document: $X = (X_1, \dots, X_p)^\top$ is a random vector, Σ is its **covariance matrix**, α is a unit direction, and $Y_1 = \alpha^\top X$ is the first principal component. Read Σ as the C of the earlier sections and α as w ; nothing else changes.

Total: 40 marks. Work each one on paper first — the solutions are written out in full precisely so that you can compare your method, not just your answer.

Question 1 (6 marks) from raw data to components

The following four observations are recorded on two variables:

$$(2, 3), \quad (4, 7), \quad (6, 5), \quad (8, 9).$$

- Compute the mean vector, write down the centred data matrix, and hence obtain the covariance matrix Σ (divide by N). [2 Marks]
- Determine the eigenvalues of Σ and the corresponding *normalised* eigenvectors. [2 Marks]
- Obtain the first principal component score for each observation, and verify that $\text{Var}(Y_1) = \lambda_1$. State the proportion of total variance explained by Y_1 . [2 Marks]

Solution 1

How to attack this type. Always the same five moves: mean $\rightarrow$ centre $\rightarrow \frac{1}{N} X^\top X \rightarrow$ characteristic polynomial $\rightarrow$ normalise. Check the centred columns sum to zero before going further; if they do not, everything after is wrong. Use tr and det to check the eigenvalues.

(a) Means: $\mu_1 = \frac{2+4+6+8}{4} = 5$, $\mu_2 = \frac{3+7+5+9}{4} = 6$. Subtracting (5, 6):

$$X = \begin{pmatrix} -3 & -3 \\ -1 & 1 \\ 1 & -1 \\ 3 & 3 \end{pmatrix}, \quad \text{columns sum to 0.} \checkmark$$

$$\sum x_1^2 = 9 + 1 + 1 + 9 = 20, \quad \sum x_2^2 = 20, \quad \sum x_1 x_2 = 9 - 1 - 1 + 9 = 16.$$

$$\Sigma = \frac{1}{4} X^\top X = \frac{1}{4} \begin{pmatrix} 20 & 16 \\ 16 & 20 \end{pmatrix} = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}.$$

(b) Characteristic equation:

$$\det(\Sigma - \lambda I) = (5 - \lambda)^2 - 16 = 0 \implies 5 - \lambda = \pm 4 \implies \lambda_1 = 9, \quad \lambda_2 = 1.$$

Check: $\lambda_1 + \lambda_2 = 10 = \text{tr}(\Sigma)$, $\lambda_1 \lambda_2 = 9 = \det(\Sigma) = 25 - 16$. Both hold.

For $\lambda_1 = 9$: $(5 - 9)a + 4b = 0 \implies -4a + 4b = 0 \implies a = b$, giving (1, 1) of length $\sqrt{2}$, so

$$\alpha_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

For $\lambda_2 = 1$: $(5 - 1)a + 4b = 0 \Rightarrow 4a + 4b = 0 \Rightarrow b = -a$, so

$$\alpha_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Check: $\alpha_1^\top \alpha_2 = (1 - 1)/2 = 0$, and both have length 1.

(c) $Y_1 = \frac{1}{\sqrt{2}}(x_1 + x_2)$ applied to the centred rows:

$$\frac{-3-3}{\sqrt{2}} = -\frac{6}{\sqrt{2}} = -3\sqrt{2} \approx -4.243, \quad \frac{-1+1}{\sqrt{2}} = 0, \quad \frac{1-1}{\sqrt{2}} = 0, \quad \frac{3+3}{\sqrt{2}} = 3\sqrt{2} \approx 4.243.$$

The scores sum to zero, so their variance is the mean of their squares:

$$\text{Var}(Y_1) = \frac{18 + 0 + 0 + 18}{4} = \frac{36}{4} = 9 = \lambda_1. \checkmark$$

Proportion explained:

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{9}{10} = 90\%.$$

Question 2 (6 marks) find the faults in a derivation

A student is asked to obtain the first principal component of a two-dimensional random vector $X = (X_1, X_2)^\top$ with

$$\Sigma = \begin{pmatrix} 5 & 2 \\ 2 & 8 \end{pmatrix},$$

using Lagrange multipliers. Their complete attempt is reproduced below.

- (1) Maximise $\text{Var}(Y_1) = \alpha^\top \Sigma \alpha$ subject to $\alpha^\top \alpha = 1$.
- (2) $\mathcal{L}(\alpha, \lambda) = \alpha^\top \Sigma \alpha - \lambda(\alpha^\top \alpha - 1)$.
- (3) $\frac{\partial \mathcal{L}}{\partial \alpha} = 2\Sigma \alpha - \lambda \alpha = \mathbf{0}$, so $\Sigma \alpha = \frac{1}{2} \lambda \alpha$.
- (4) $\det(\Sigma - \lambda I) = \lambda^2 - 13\lambda + 36 = 0$, so $\lambda = 9$ and $\lambda = 4$.
- (5) For the larger root, $(\Sigma - 9I)\alpha = \mathbf{0}$ gives $\alpha = (1, 2)^\top$, so $Y_1 = X_1 + 2X_2$.
- (6) $\text{Var}(Y_1) = 9$, because the maximum variance is the largest eigenvalue.

Exactly three of the six lines are faulty.

- (a) Identify the three faulty lines and state, in one sentence each, what is wrong. [3 Marks]
- (b) Rewrite the derivation correctly from the first faulty line onwards, and give the correct Y_1 and $\text{Var}(Y_1)$. [2 Marks]
- (c) Line (6) contradicts the student's own line (5). Compute $\text{Var}(X_1 + 2X_2)$ directly from Σ to show this, and state the general relation between $\text{Var}(\alpha^\top X)$ and λ when α is an eigenvector that has *not* been normalised. [1 Mark]

Solution 2

How to attack this type. Marking someone else's work is the fastest way to learn a derivation, because you cannot pattern-match your way through it. Walk the six lines in

order against the three questions that matter at each step: is the Lagrangian written with the constraint in the form $(\cdot) = 0$? is every derivative right? and does the final vector satisfy $\|\alpha\| = 1$? Any answer that survives all three is almost certainly correct.

(a) Lines (1), (2) and (4) are correct. The three faults are:

- (3) **The derivative of the constraint term is wrong.** $\partial(\alpha^\top \alpha)/\partial \alpha = 2\alpha$, not α , so the second term should be $-2\lambda\alpha$. As written, the student's own algebra makes the multiplier *twice* the eigenvalue, which destroys the identity $\text{Var}(Y_1) = \lambda$ they use in line (6).
- (5) **The eigenvector has not been normalised.** $(1, 2)^\top$ is the right *direction*, but its length is $\sqrt{5}$, and the whole derivation assumed $\alpha^\top \alpha = 1$.
- (6) **The value is wrong, given line (5).** With the unnormalised α of line (5) the variance is 45, not 9; $\text{Var} = \lambda$ holds only for a unit eigenvector.

(b) Corrected from line (3) onwards. Both terms of $\mathcal{L}$ are quadratic forms, so

$$\frac{\partial \mathcal{L}}{\partial \alpha} = 2\Sigma\alpha - 2\lambda\alpha = 0 \quad \implies \quad \Sigma\alpha = \lambda\alpha.$$

Line (4) stands: $\det(\Sigma - \lambda I) = (5 - \lambda)(8 - \lambda) - 4 = \lambda^2 - 13\lambda + 36 = 0$, so $\lambda = \frac{13 \pm 5}{2}$, giving $\lambda_1 = 9$ and $\lambda_2 = 4$. *Check: $9 + 4 = 13 = \text{tr}(\Sigma)$ and $9 \times 4 = 36 = \det(\Sigma) = 40 - 4$. Both hold.*

For $\lambda_1 = 9$: $(5 - 9)\alpha_1 + 2\alpha_2 = 0 \Rightarrow \alpha_2 = 2\alpha_1$, so the direction is $(1, 2)^\top$, of length $\sqrt{5}$. Normalising,

$$\alpha_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \boxed{Y_1 = \frac{X_1 + 2X_2}{\sqrt{5}}}, \quad \text{Var}(Y_1) = \lambda_1 = 9.$$

(For completeness, $\lambda_2 = 4$ gives $(5 - 4)\alpha_1 + 2\alpha_2 = 0$, so $\alpha_2 = \frac{1}{\sqrt{5}}(-2, 1)^\top$; note $\alpha_1^\top \alpha_2 = (-2 + 2)/5 = 0$.) The first component explains $9/13 = 69.2\%$ of the total variance.

(c) Compute the variance of the student's $Y_1 = X_1 + 2X_2$ directly:

$$\text{Var}(X_1 + 2X_2) = \text{Var}(X_1) + 4\text{Var}(X_2) + 4\text{Cov}(X_1, X_2) = 5 + 4(8) + 4(2) = 45,$$

or equivalently $(1, 2)\Sigma(1, 2)^\top = 45$ — five times the claimed 9. In general, if $\Sigma\alpha = \lambda\alpha$ then

$$\text{Var}(\alpha^\top X) = \alpha^\top \Sigma \alpha = \lambda \alpha^\top \alpha = \lambda \|\alpha\|^2,$$

so the tidy statement $\text{Var} = \lambda$ is true *only* for a unit vector. Here $\|\alpha\|^2 = 5$, and indeed $5 \times 9 = 45$.

Now extend it. Run the corrected derivation again with a general $p \times p$ covariance matrix and no numbers at all, then a third time for the second component, carrying the extra constraint $\alpha_2^\top \alpha_1 = 0$. §4 has the model answer for both if you get stuck — but write yours first.

Question 3 (5 marks) projecting a new observation

A PCA is fitted on a two-variable training set. The training means are $\mu = (4, 10)^\top$ and the training standard deviations are $\sigma = (2, 5)^\top$. On the standardised data the retained

component is

$$\alpha_1 = \begin{pmatrix} 3/5 \\ 4/5 \end{pmatrix}, \quad \text{and the discarded one is} \quad \alpha_2 = \begin{pmatrix} -4/5 \\ 3/5 \end{pmatrix}.$$

A new observation $x_* = (6, 20)^\top$ arrives.

- (a) Standardise x_* , stating clearly which mean and standard deviation you use and why.

[1 Mark]

- (b) Compute the score $z_* = \alpha_1^\top x_*$ and the reconstruction $\tilde{x}_* = \alpha_1 \alpha_1^\top x_*$ (both on the standardised scale). [2 Marks]

- (c) Compute the squared reconstruction error, and verify that it equals the squared score along the discarded direction. [2 Marks]

Solution 3

How to attack this type. The only conceptual point is that μ and σ come from training. After that it is arithmetic. For part (c), do not compute the residual the long way twice — get it once by subtraction, then confirm it with z_2^2 ; the two must agree because α_1 and α_2 form an orthonormal basis and Pythagoras applies.

- (a) Use the *training* statistics — a single new point has no mean or spread of its own, and using test statistics would leak information from the test set into the model:

$$x_*^{\text{std}} = \begin{pmatrix} (6-4)/2 \\ (20-10)/5 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

- (b) The score is a single dot product:

$$z_* = \alpha_1^\top x_*^{\text{std}} = \frac{3}{5}(1) + \frac{4}{5}(2) = \frac{3}{5} + \frac{8}{5} = \frac{11}{5} = 2.2.$$

The reconstruction puts that number back along α_1 :

$$\tilde{x}_* = z_* \alpha_1 = 2.2 \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix} = \begin{pmatrix} 1.32 \\ 1.76 \end{pmatrix}.$$

Note $x_*^{\text{std}} = (1, 2)$ but $\tilde{x}_* = (1.32, 1.76)$: the reconstruction is close, not exact, because we kept only one of the two directions.

- (c) The residual is what did not survive:

$$r = x_*^{\text{std}} - \tilde{x}_* = \begin{pmatrix} 1 - 1.32 \\ 2 - 1.76 \end{pmatrix} = \begin{pmatrix} -0.32 \\ 0.24 \end{pmatrix}, \quad \|r\|^2 = 0.1024 + 0.0576 = \boxed{0.16}.$$

Now along the discarded direction:

$$z_2 = \alpha_2^\top x_*^{\text{std}} = -\frac{4}{5}(1) + \frac{3}{5}(2) = -\frac{4}{5} + \frac{6}{5} = \frac{2}{5} = 0.4, \quad z_2^2 = 0.16. \checkmark$$

They agree. This is Pythagoras in the orthonormal basis $\{\alpha_1, \alpha_2\}$: since $\|x\|^2 = z_1^2 + z_2^2$, whatever you do not keep is exactly what shows up as error. Check the basis first if the two numbers disagree: $\alpha_1^\top \alpha_2 = -\frac{12}{25} + \frac{12}{25} = 0$ and both have length $\sqrt{(3/5)^2 + (4/5)^2} = 1$.

Question 4 (6 marks) three variables, and using structure

A three-variable random vector $X = (X_1, X_2, X_3)^T$ has covariance matrix

$$\Sigma = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 4 & 0 \\ 0 & 0 & 3 \end{pmatrix}.$$

- Without expanding the full cubic $\det(\Sigma - \lambda I) = 0$, explain what the pattern of zeros in Σ tells you about X_3 , and hence write down all three eigenvalues. Verify your list against $\text{tr}(\Sigma)$ and $\det(\Sigma)$. [2 Marks]
- Obtain the three normalised eigenvectors and confirm that they are mutually orthogonal. [2 Marks]
- Write down Y_1 explicitly and state $\text{Var}(Y_1)$. Then show that if only Y_1 and Y_2 are kept, X_3 is reconstructed exactly while X_1 and X_2 are not; identify the quantity that is lost and verify that its variance is λ_3 . [2 Marks]

Solution 4

How to attack this type. Look at the shape of the matrix before you start grinding. Zeros in a covariance matrix mean uncorrelated variables, and a variable uncorrelated with all the others splits off into a block of its own: its variance is an eigenvalue outright, and its axis is an eigenvector outright. That turns a cubic into a quadratic plus a freebie. When you do reach the small block, it is the ordinary 2×2 routine — characteristic polynomial, then $(\Sigma - \lambda I)\alpha = 0$, then normalise.

(a) The third row and column are zero apart from the diagonal, so $\text{Cov}(X_3, X_1) = \text{Cov}(X_3, X_2) = 0$: X_3 is **uncorrelated with the other two**. Σ is therefore block diagonal,

$$\Sigma = \begin{pmatrix} A & \mathbf{0} \\ \mathbf{0} & 3 \end{pmatrix}, \quad A = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix},$$

and the eigenvalues of a block-diagonal matrix are simply the eigenvalues of its blocks pooled together. The 1×1 block contributes $\lambda = 3$. For A ,

$$\det(A - \lambda I) = (4 - \lambda)^2 - 4 = 0 \implies 4 - \lambda = \pm 2 \implies \lambda = 6 \text{ and } \lambda = 2.$$

Sorted: $\lambda_1 = 6, \lambda_2 = 3, \lambda_3 = 2$. Check: $6 + 3 + 2 = 11 = 4 + 4 + 3 = \text{tr}(\Sigma)$, and $6 \times 3 \times 2 = 36$, while $\det(\Sigma) = 3 \times \det(A) = 3(16 - 4) = 36$. Both hold.

(b) Because X_3 is decoupled, two of the eigenvectors lie entirely in the (X_1, X_2) plane and the third is the X_3 axis itself.

For $\lambda_1 = 6$, inside A : $(4 - 6)\alpha_1 + 2\alpha_2 = 0 \implies \alpha_1 = \alpha_2$, giving $(1, 1)$; embedded in three dimensions and normalised,

$$\alpha_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}.$$

For $\lambda_2 = 3$, the block is already diagonal, so

$$\alpha_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

For $\lambda_3 = 2$: $(4 - 2)\alpha_1 + 2\alpha_2 = 0 \Rightarrow \alpha_2 = -\alpha_1$, giving

$$\alpha_3 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix}.$$

Orthogonality: $\alpha_1^\top \alpha_3 = \frac{1}{2}(1 - 1 + 0) = 0$, and both $\alpha_1^\top \alpha_2 = 0$ and $\alpha_3^\top \alpha_2 = 0$ because α_2 has zeros in the first two slots. All three have length 1. *Spot check*: $\Sigma \alpha_1 = \frac{1}{\sqrt{2}}(4 + 2, 2 + 4, 0)^\top = \frac{6}{\sqrt{2}}(1, 1, 0)^\top = 6\alpha_1$. ✓

(c) The first principal component uses the eigenvector of the largest eigenvalue:

$$Y_1 = \alpha_1^\top X = \frac{X_1 + X_2}{\sqrt{2}}, \quad \text{Var}(Y_1) = \lambda_1 = 6.$$

Keeping Y_1 and Y_2 means $W = [\alpha_1 \mid \alpha_2]$, and the reconstruction of any point is $\tilde{x} = WW^\top x$. Build that projector:

$$WW^\top = \alpha_1 \alpha_1^\top + \alpha_2 \alpha_2^\top = \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Applying it to $x = (x_1, x_2, x_3)^\top$:

$$\tilde{x} = \left(\frac{x_1 + x_2}{2}, \frac{x_1 + x_2}{2}, x_3 \right)^\top.$$

The third row of the projector is $(0, 0, 1)$, so $\tilde{x}_3 = x_3$ **exactly**. But x_1 and x_2 are both replaced by their average, so neither survives unless they happened to be equal.

What was lost is the residual

$$x - \tilde{x} = \left(\frac{x_1 - x_2}{2}, -\frac{x_1 - x_2}{2}, 0 \right)^\top = \frac{x_1 - x_2}{2} (1, -1, 0)^\top,$$

which points along the discarded direction α_3 . So the quantity thrown away is the **difference** $X_1 - X_2$, and its score is $Y_3 = \alpha_3^\top X = (X_1 - X_2)/\sqrt{2}$. Its variance:

$$\text{Var}\left(\frac{X_1 - X_2}{\sqrt{2}}\right) = \frac{\text{Var}(X_1) + \text{Var}(X_2) - 2\text{Cov}(X_1, X_2)}{2} = \frac{4 + 4 - 2(2)}{2} = 2 = \lambda_3. \checkmark$$

PCA kept the *sum* of the correlated pair and discarded their *difference* — which is exactly right, since the pair move together and the difference is the small, quiet part of their joint variation.

Worth noticing: Y_2 is just X_3 on its own. A variable uncorrelated with all the others contributes its variance to the spectrum directly, and here, because 3 is not equal to 6 or 2, its axis comes back as a principal component untouched. PCA only mixes variables that actually move together. (Had Σ_{33} been 6, the repeated eigenvalue would have made that component non-unique — an honest exception to the rule.)

Now extend it. Change the $(3,3)$ entry from 3 to 7 and redo the question — which variable becomes the first principal component, and does anything about part (c) change? Then take $\Sigma = \text{diag}(4, 5, 7)$, with no correlations at all, and say what PCA does to a covariance matrix that is already diagonal.

Question 5 (6 marks) PCA when $D \gg N$

A gene-expression study records $D = 12,000$ genes on $N = 40$ tissue samples. The centred data matrix X has size $40 \times 12,000$ (rows are samples), and the covariance matrix is $\Sigma = \frac{1}{N} X^\top X$.

- State the size of Σ and the memory required to hold it in 8-byte doubles. Explain why a direct eigendecomposition is impractical. [2 Marks]
- Show that if $\Sigma w = \lambda w$, then $u = Xw$ satisfies $Ku = \lambda u$, where $K = \frac{1}{N} XX^\top$. State the size of K . [2 Marks]
- Given an eigenvector u of K , recover the corresponding eigenvector of Σ and state the normalisation required. What is the largest possible number of non-zero eigenvalues of Σ ? [2 Marks]

Solution 5

How to attack this type. Size questions first — the covariance matrix is always (number of features) $\times$ (number of features), whatever N is. The derivation in (b) is one move: left-multiply the eigen-equation by X and regroup the brackets. In (c) remember that a proportionality is not an answer until you normalise, and remember the rank argument — centring costs one degree of freedom.

(a) Σ is $D \times D = 12,000 \times 12,000$, which is 1.44×10^8 entries. At 8 bytes each:

$$1.44 \times 10^8 \times 8 = 1.152 \times 10^9 \text{ bytes} \approx 1.15 \text{ GB.}$$

Two problems. First, storage — over a gigabyte for a raw dataset of a few megabytes. Second, and worse, eigendecomposition costs about $O(D^3)$ operations, which here is $\approx 1.7 \times 10^{12}$ — hours of computation to extract structure from only 40 data points.

(b) Begin from the eigen-equation and left-multiply by X :

$$\frac{1}{N} X^\top X w = \lambda w \xrightarrow{\times X} \frac{1}{N} X X^\top X w = \lambda X w.$$

Regroup the left-hand side as $(\frac{1}{N} X X^\top)(Xw)$ and set $u = Xw$:

$$\boxed{Ku = \lambda u}, \quad K = \frac{1}{N} X X^\top.$$

K is $N \times N = 40 \times 40$ — ninety thousand times fewer entries than Σ , and the eigenvalues λ are identical.

(c) Left-multiply $Ku = \lambda u$ by $X^\top$:

$$X^\top Ku = \lambda X^\top u \implies \frac{1}{N} X^\top X X^\top u = \lambda X^\top u \implies \Sigma(X^\top u) = \lambda(X^\top u).$$

So $X^\top u$ is an eigenvector of Σ for the same λ . It is not a unit vector, so normalise:

$$\boxed{w = \frac{X^\top u}{\|X^\top u\|}}$$

For the count: $\text{rank}(X) \leq \min(N, D) = 40$, and centring the rows imposes one linear relation among them, so $\text{rank}(X) \leq 39$ and therefore

$$\text{rank}(\Sigma) \leq N - 1 = 39.$$

At most 39 of the 12,000 eigenvalues can be non-zero; the rest are exactly zero. So working with the 40×40 matrix K loses nothing — it already contains every non-zero eigenvalue that exists.

Question 6 (5 marks) eigenvalues, variance shares, and decorrelation

The covariance matrix Σ of a five-variable dataset has eigenvalues

$$\lambda_1 = 4.5, \quad \lambda_2 = 2.1, \quad \lambda_3 = 0.9, \quad \lambda_4 = 0.4, \quad \lambda_5 = 0.1.$$

- State the total variance in the data and the value of $\text{tr}(\Sigma)$. [1 Mark]
- Find the proportion of variance explained by the first two components, and the smallest number of components needed to retain at least 90%. [2 Marks]
- Let W hold all five orthonormal eigenvectors as columns and let $Z = XW$ be the component scores. Show that $\text{Cov}(Z)$ is diagonal, and state its diagonal entries. What does this tell you about the components? [2 Marks]

Solution 6

How to attack this type. Part (a) is the trace identity, which is worth a mark for one line of work — do not skip it. In (b) build the cumulative list once and read both answers off it. Part (c) is a three-line proof; the only facts you need are $\Sigma W = W\Lambda$ and $W^\top W = I$.

(a) Rotating to the principal axes does not change the total spread, so the total variance is the sum of the eigenvalues, and this equals the trace:

$$\text{tr}(\Sigma) = \sum_{d=1}^5 \lambda_d = 4.5 + 2.1 + 0.9 + 0.4 + 0.1 = 8.0.$$

(b) Individual and cumulative shares:

component	1	2	3	4	5
λ_d	4.5	2.1	0.9	0.4	0.1
share (%)	56.25	26.25	11.25	5.00	1.25
cumulative (%)	56.25	82.50	93.75	98.75	100.00

The first two components explain

$$\frac{4.5 + 2.1}{8.0} = \frac{6.6}{8.0} = 82.5\%.$$

Two components fall short of 90%; three reach 93.75%. So $M = 3$ components are required.

(c) X is centred, so $Z = XW$ is centred too, and

$$\text{Cov}(Z) = \frac{1}{N} Z^\top Z = \frac{1}{N} (XW)^\top (XW) = W^\top \left(\frac{X^\top X}{N} \right) W = W^\top \Sigma W.$$

The eigen-equations for all five vectors written together give $\Sigma W = W\Lambda$, and W is orthogonal, so $W^\top W = I$. Therefore

$$\text{Cov}(Z) = W^\top W \Lambda = \Lambda = \text{diag}(4.5, 2.1, 0.9, 0.4, 0.1).$$

Every off-diagonal entry is zero. Off-diagonal entries of a covariance matrix are the covariances between different components, so **the principal components are mutually uncorrelated**. The diagonal entries confirm once more that the variance of the d th component is λ_d .

Practically: the scores can be fed straight into a regression without any multicollinearity between them — which is one of the main reasons PCA is used as a preprocessing step at all.

Question 7 (6 marks) from singular values to components

The centred data matrix X has $N = 100$ rows and $D = 4$ columns, and its singular values are

$$\sigma_1 = 20, \quad \sigma_2 = 10, \quad \sigma_3 = 5, \quad \sigma_4 = 1.$$

- (a) Obtain the eigenvalues of $\Sigma = \frac{1}{N} X^\top X$, and state where the principal directions appear in the decomposition $X = U \Sigma_s V^\top$. [2 Marks]
- (b) Find the proportion of variance explained by the first two components, and the smallest number of components retaining at least 95%. [2 Marks]
- (c) Write down the best rank-2 approximation of X , and compute the resulting error in both the spectral and the Frobenius norm. [2 Marks]

Here Σ_s denotes the diagonal matrix of singular values, to keep it distinct from the covariance matrix Σ .

Solution 7

How to attack this type. One formula does part (a): $\lambda_d = \sigma_d^2 / N$. Take care over which of U and V carries the directions — the one whose rows are indexed by features. Part (c) is Eckart–Young: truncate, and the leftover error is built from the singular values you dropped.

(a) From $X = U \Sigma_s V^\top$,

$$\Sigma = \frac{1}{N} X^\top X = V \left(\frac{\Sigma_s^2}{N} \right) V^\top \implies \lambda_d = \frac{\sigma_d^2}{N}.$$

With $N = 100$:

$$\lambda_1 = \frac{400}{100} = 4, \quad \lambda_2 = \frac{100}{100} = 1, \quad \lambda_3 = \frac{25}{100} = 0.25, \quad \lambda_4 = \frac{1}{100} = 0.01.$$

The principal directions are the columns of V — the right singular vectors — because with data in rows it is V that has $D = 4$ rows and so lives in feature space. (Had the data been stored in columns, they would be the columns of U .)

(b) Total variance = $4 + 1 + 0.25 + 0.01 = 5.26$.

$$\text{first two: } \frac{4 + 1}{5.26} = \frac{5}{5.26} = 95.06\%.$$

The first component alone gives $4/5.26 = 76.05\%$, which is below 95%, and two components give 95.06%, which clears it. So $\boxed{M = 2}$.

(c) By the Eckart–Young theorem the best rank-2 approximation is the SVD truncated after two terms:

$$\tilde{X}_2 = U_2 \Sigma_{s,2} V_2^\top = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^\top,$$

using the first two columns of U and V and the two largest singular values. The errors are made entirely of the discarded singular values:

$$\|X - \tilde{X}_2\|_2 = \sigma_3 = 5, \quad \|X - \tilde{X}_2\|_F = \sqrt{\sigma_3^2 + \sigma_4^2} = \sqrt{25 + 1} = \sqrt{26} \approx 5.10.$$

No rank-2 matrix, however cleverly constructed, gets closer to X than this one.

What the seven questions were testing

- Q1 the full computation, from raw numbers to scores
- Q2 auditing a derivation — knowing it well enough to find a fault in it
- Q3 applying a fitted PCA to new data, and where the error goes
- Q4 reading structure out of a matrix before computing anything
- Q5 what changes when there are more columns than rows
- Q6 eigenvalues as a variance budget, and why components decorrelate
- Q7 the SVD route, and best low-rank approximation

If a question type gave you trouble, the matching section is §12 (Q1), §4 (Q2, Q4), §11 (Q3), §9 (Q5), §10 (Q6) and §6–§7 (Q7).